PRACA DYPLOMOWA

WYDZIAŁ
BUDOWY MASZYN I INFORMATYKI
KIERUNEK: INFORMATYKA
SPECJALNOŚĆ: INŻYNIERIA OPROGRAMOWANIA

Radosław Skrzypczyński

nr albumu: 29429

Praca inżynierska

Projekt: Gra wokalna karaoke z wykorzystaniem sztucznej inteligencji

Kategoria pracy: projektowa

Promotor: dr MARCIN BERNAŚ

Bielsko-Biała, r. ak. 2025/2026

Spis treści

Rozdział 1 — Wstęp **4**

1.1. Wprowadzenie 4

1.2. Cel i zakres pracy 4

1.3. Motywacja 5

1.4. Struktura pracy 7

Rozdział 2 — Analiza literatury i metody **7**

2.1. Historia i kultura karaoke 7

2.2. Przegląd literatury naukowej z zakresu analizy śpiewu 8

2.3. Metody detekcji wysokości dźwięku w literaturze 9

2.4. Separacja źródeł audio w kontekście karaoke 10

2.5. Analiza istniejących aplikacji karaoke 11

2.6. Wnioski z analizy — luka w istniejących rozwiązaniach 16

2.7. Cyfrowe przetwarzanie sygnałów audio — podstawy 18

2.8. Ekstrakcja cech akustycznych 19

2.9. Detekcja wysokości dźwięku (pitch detection) 20

2.10. Dopasowanie czasowe — Dynamic Time Warping (DTW) 22

2.12. Format UltraStar TXT 23

Rozdział 3 — Projekt i architektura systemu **24**

3.1. Wymagania funkcjonalne i niefunkcjonalne 24

3.2. Architektura systemu 25

3.3. Model danych 26

3.4. Stos technologiczny 28

3.5. Komunikacja w czasie rzeczywistym — ogólna koncepcja 29

Rozdział 4 — Implementacja **29**

4.1. Backend — AudioVerse.API (.NET 10) 29

4.2. Mikroserwisy AI (Python / FastAPI) 36

4.3. Frontend — interfejs użytkownika (React + TypeScript) 37

4.4. Silnik karaoke — timeline i detekcja pitch w przeglądarce 42

4.5. Ocena śpiewu — kryteria i metryki 43

4.6. Generowanie raportów PDF — LaboratoryReportPdfService 44

4.7. Wdrożenie — Docker Compose 45

Rozdział 5 — Testy, analiza wyników i podsumowanie 47

5.1. Metodologia testowania 47

5.2. Testy funkcjonalne systemu 47

5.3. Porównanie algorytmów detekcji wysokości dźwięku 49

5.4. Ocena jakości systemu oceniania 51

5.5. Ograniczenia i kierunki dalszego rozwoju 52

5.6. Podsumowanie i wnioski 53

Rozdział 1 — Wstęp

1.1. Wprowadzenie

Muzyka od zawsze pełniła istotną rolę w życiu człowieka, stanowiąc formę sztuki, środek wyrażania emocji, narzędzie integracji społecznej oraz źródło rozrywki. W epoce cyfrowej dostępność muzyki zasadniczo się zmieniła: platformy streamingowe, interfejsy programistyczne i powszechny dostęp do sieci internetowej sprawiają, że słuchacze stali się zarazem aktywnymi uczestnikami tworzenia muzyki. Jedną z popularnych form aktywnego angażowania odbiorców w świat muzyki jest karaoke — rozrywka polegająca na śpiewaniu piosenek przy akompaniamencie podkładu muzycznego, często wzbogaconego o wyświetlany w czasie rzeczywistym tekst i nuty[1].

Tradycyjne systemy karaoke ograniczają się do dostarczania ścieżki instrumentalnej i tekstu, pozostawiając ocenę wykonania subiektywnym odczuciom uczestników. Algorytmy detekcji podstawowej częstotliwości (F0, ang. *fundamental frequency estimation*), separacji źródeł audio i dynamicznego dopasowywania czasowego (DTW, ang. *Dynamic Time Warping*)[2] oferują dziś dokładność wystarczającą do automatycznej, interaktywnej oceny jakości śpiewu w czasie rzeczywistym[3],[4]. Dzięki temu karaoke może pełnić funkcję trenera wokalnego — z obiektywną informacją zwrotną per nuta, a nie wyłącznie rozrywką. Funkcję rozrywkową dzięki temu można wzbogacić o element współzawodnictwa.

1.2. Cel i zakres pracy

Celem niniejszej pracy inżynierskiej jest zaprojektowanie, implementacja i ocena systemu gry wokalnej karaoke wykorzystującej metody sztucznej inteligencji do analizy i oceny śpiewu użytkownika. System ma stanowić połączenie tradycyjnej rozrywki karaoke

---

[1] T. Mitsui, S. Hosokawa (red.), Karaoke Around the World: Global Technology, Local Singing, Routledge, 1998.

[2] H. Sakoe, S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition", IEEE Trans. ASSP, vol. 26, nr 1, s. 43–49, 1978.

[3] M. Müller, Fundamentals of Music Processing: Using Python and Jupyter Notebooks, wyd. 2, Springer, 2021.

[4] J. W. Kim, J. Salamon, P. Li, J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation", Proc. ICASSP, 2018.

z inteligentnym mechanizmem oceny, dostarczającym użytkownikowi natychmiastowej informacji zwrotnej na temat poprawności intonacji i rytmu w formie punktów.

Zakres pracy obejmuje:

- analizę literatury naukowej dotyczącej automatycznej analizy śpiewu, algorytmów detekcji F0 oraz separacji źródeł audio;
- przegląd i porównawczą ocenę istniejących aplikacji karaoke pod kątem wykorzystania metod sztucznej inteligencji;
- dobór i opis metod AI zastosowanych w systemie: pYIN i CREPE do detekcji F0 oraz DTW do porównawczej oceny śpiewu;
- zaprojektowanie i implementację architektury mikroserwisowej systemu AudioVerse, obejmującej warstwę frontendową (React/TypeScript), backendową (ASP.NET Core/.NET 10) oraz mikroserwisy AI (Python/FastAPI);
- implementację dedykowanego modułu laboratoryjnego (LaboratoryController) umożliwiającego pomiarowe testy algorytmów detekcji F0;
- weryfikację działania algorytmów pYIN i CREPE dla plików wokalnych z różnym poziomem akompaniamentu i pomiar latencji mikroserwisów AI;
- ocenę spełnienia wymagań niefunkcjonalnych i sformułowanie zaleceń dotyczących doboru algorytmu do zastosowań webowych.

Praca skupia się wyłącznie na aspektach muzycznych — intonacji i rytmie — i nie obejmuje rozpoznawania mowy ani detekcji dykcji. Wszystkie eksperymenty i testy opisane w pracy przeprowadzono w trybie singleplayer.

1.3. Motywacja

Autor niniejszej pracy od wielu lat korzysta z UltraStar Deluxe — open-source'owego następcy SingStar na PC — jako podstawowej formy rozrywki karaoke. To właśnie regularne sesje z tym systemem uświadomiły, że automatyczna ocena intonacji może pełnić funkcję wykraczającą poza punktację: jest narzędziem rzeczywistej nauki kontroli głosu. Dobry głos nie jest warunkiem koniecznym — istotna jest informacja zwrotna w czasie rzeczywistym, para nuta po nucie, pozwalająca ćwiczyć ton z iteracyjną precyzją, której nie oferuje żadna forma

śpiewania „na sucho". Welch i in.[5] potwierdzają to eksperymentalnie: regularny trening ze sprzężeniem zwrotnym istotnie poprawia dokładność intonacji.

Pierwszym źródłem motywacji jest zatem wiarygodność tej funkcji coachingowej — i jej obecna niedostępność poza środowiskiem desktopowym. Istniejące platformy mobilne karaoke kierują rozgrywkę ku rywalizacji online, co — zdaniem autora — mija się z imprezowym, wspólnotowym charakterem karaoke: grą *couch*, w jednym pomieszczeniu, z żywą publicznością. Proponowane rozwiązanie webowe, działające w przeglądarce bez instalacji, przywraca tę formułę w nowoczesnym środowisku i dostarcza obiektywnej oceny per nuta każdemu użytkownikowi, niezależnie od sprzętu.

Drugim czynnikiem jest dostępność zaawansowanych modeli AI jako bibliotek open-source. Modele CREPE[6] i Demucs[7] można wdrożyć na serwerze z GPU klasy konsumenckiej, co kilka lat temu byłoby niemożliwe w warunkach akademickich. Biblioteka librosa udostępnia skuteczną implementację pYIN[8], co umożliwiło szybką integrację algorytmu bez implementacji od zera.

Trzecim bodźcem jest zidentyfikowana luka technologiczna: żadna z popularnych platform karaoke nie łączy detekcji F0 opartej na głębokim uczeniu z oceną DTW i wizualną informacją zwrotną w czasie rzeczywistym w ramach otwartej, webowej platformy działającej w przeglądarce bez instalacji.

Czwartym czynnikiem jest potrzeba mierzalności: brak dostępnego narzędzia do porównania pYIN i CREPE na nagraniach z akompaniamentem w warunkach latencji typowych dla aplikacji webowej. System AudioVerse zawiera dedykowany moduł pomiarowy (LaboratoryController) umożliwiający takie testy w sposób reprodukowalny i zautomatyzowany.

---

[5] G. F. Welch i in., "The Impact of Singing on Brain and Behaviour", Progress in Brain Research, vol. 237, 2018.

[6] J. W. Kim, J. Salamon, P. Li, J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation", Proc. ICASSP, 2018.

[7] A. Défossez, "Hybrid Spectrogram and Waveform Source Separation", Proc. ISMIR 2021, arXiv:2111.03600, 2021.

[8] M. Mauch, S. Dixon, "pYIN: A Fundamental Frequency Estimator Using Probabilistic Threshold Distributions", Proc. ICASSP, s. 659–663, 2014.

1.4. Struktura pracy

Praca składa się z pięciu rozdziałów merytorycznych.

- **Rozdział 1** (niniejszy) zawiera wprowadzenie, sformułowanie celów i zakresu oraz motywację podjętego tematu.
- **Rozdział 2** obejmuje analizę literatury naukowej i stosowanych metod: historię karaoke, przegląd automatycznej analizy śpiewu w literaturze, porównanie siedmiu aplikacji karaoke, a następnie opis metod stosowanych w systemie — podstawy cyfrowego przetwarzania sygnałów, algorytmy detekcji F0 (YIN, pYIN, CREPE), algorytm DTW, format UltraStar TXT oraz stos technologiczny i koncepcję komunikacji w czasie rzeczywistym.
- **Rozdział 3** opisuje projekt i architekturę systemu AudioVerse: wymagania funkcjonalne i niefunkcjonalne, architekturę mikroserwisową oraz model danych.
- **Rozdział 4** omawia implementację systemu: backend ASP.NET Core z wzorcem CQRS, mikroserwisy AI (pYIN, CREPE, DTW), interfejs użytkownika React, silnik karaoke oparty na Canvas API, moduł laboratoryjny LaboratoryController i wdrożenie w Docker Compose.
- **Rozdział 5** poświęcony jest testom i analizie wyników: testom funkcjonalnym, porównaniu algorytmów pYIN/CREPE, benchmarkowi latencji mikroserwisów AI i podsumowaniu z wnioskami.

Rozdział 2 — Analiza literatury i metody

2.1. Historia i kultura karaoke

Karaoke (jap. カラオケ, od *kara* — puste, *ōkesutora* — orkiestra) powstało w Japonii na początku lat siedemdziesiątych XX wieku. Za twórcę pierwszej maszyny karaoke uważa się muzyka Daisuke Inoue, który w 1971 roku skonstruował urządzenie Juke 8 — magnetofon z wzmacniaczem i wrzutnikiem monet odtwarzający nagrania podkładów muzycznych bez linii

wokalnej[9]. Wynalazek szybko rozprzestrzenił się po Japonii, stając się symbolem kultury rozrywkowej, a w dekadzie lat osiemdziesiątych dotarł do Azji Południowo-Wschodniej, Korei i USA.

Kultura karaoke nabrała wymiaru socjologicznego: bary karaoke stały się przestrzenią odreagowania stresu zawodowego w społeczeństwach silnie zhierarchizowanych, a śpiew grupowy — narzędziem budowania relacji (jap. *ningenkanke no tsukuri*)[10]. Drew[11] opisuje kulturę karaoke na Zachodzie jako „hybrydowy performans, w którym nieprofesjonalni wykonawcy świadomie naśladują gwiazdy, doświadczając tymczasowej tożsamości artystycznej" (tłum. własne).

Pojawienie się technologii MIDI i płyt LaserDisc w latach osiemdziesiątych umożliwiło synchronizację tekstu z muzyką na ekranie telewizora. Platforma PlayStation w latach 2004–2008 przyniosła przełom: gry SingStar (Sony) i Rock Band (Harmonix) wprowadziły pitch tracking mikrofonu w czasie rzeczywistym[12], odpowiednio popularyzując karaoke jako gameplay, a nie wyłącznie rozrywkę barową. Współcześnie karaoke funkcjonuje w postaci aplikacji mobilnych, platform streamingowych i webowych, obsługując dziesiątki milionów użytkowników miesięcznie.

2.2. Przegląd literatury naukowej z zakresu analizy śpiewu

Automatyczna analiza śpiewu (ang. *Automatic Singing Analysis*) stanowi subdyscyplinę muzykologii obliczeniowej (ang. *Music Information Retrieval*, MIR). Fundamentalnym podręcznikiem z tej dziedziny jest monografia Müllera *Fundamentals of Music Processing*[13], omawiająca reprezentacje czasowo- częstotliwościowe, synchronizację muzyczną i ekstrakcję cech akustycznych. Lerch[14] w podręczniku *An Introduction to Audio Content Analysis* dostarcza systematycznego przeglądu metod ekstrakcji cech od MFCC po chromagram, definiując terminologię stosowaną w niniejszej pracy.

---

[9] T. Mitsui, S. Hosokawa (red.), Karaoke Around the World: Global Technology, Local Singing, Routledge, 1998.
[10] T. Mitsui, S. Hosokawa (red.), Karaoke Around the World: Global Technology, Local Singing, Routledge, 1998.
[11] R. Drew, Karaoke Nights: An Ethnographic Rhapsody, AltaMira Press, 2001.
[12] R. Drew, Karaoke Nights: An Ethnographic Rhapsody, AltaMira Press, 2001.
[13] M. Müller, Fundamentals of Music Processing: Using Python and Jupyter Notebooks, wyd. 2, Springer, 2021.
[14] A. Lerch, An Introduction to Audio Content Analysis: Applications in Signal Processing and Music Informatics, Wiley, 2012.

Welch i in.[15] przeprowadzili metaanalizę badań nad rozwojem głosu i wokalem amatorskim, wykazując, że regularny trening ze sprzężeniem zwrotnym istotnie poprawia dokładność intonacji. Wynik ten stanowi naukowe uzasadnienie dla funkcji coachingowej aplikacji karaoke z automatyczną oceną.

Humphrey i in.[16] zaproponowali zunifikowaną platformę MIR, w której cechy akustyczne są uczone przez sieć głęboką zamiast projektowane ręcznie. Koncepcja ta znalazła bezpośrednie odzwierciedlenie m.in. w architekturze CREPE[17], który również operuje bezpośrednio na surowej fali PCM bez wstępnej ekstrakcji MFCC ani spektrogramu.

2.3. Metody detekcji wysokości dźwięku w literaturze

Detekcja podstawowej częstotliwości F0 (ang. *pitch detection* lub *fundamental frequency estimation*) jest centralnym zagadnieniem technicznym niniejszej pracy. Poniżej zestawiono ewolucję metod w kolejności chronologicznej.

Metody autokorelacyjne (lata 70.–90.) wyznaczają F0 jako odwrotność okresu sygnału, identyfikowanego przez szczyt funkcji autokorelacji. Choć proste i szybkie, są wrażliwe na wielodźwięki i oktawowe błędy (ang. *octave errors*).

Algorytm YIN (de Cheveigné i Kawahara, 2002)[18] opiera się na funkcji różnicowej (ang. *difference function*) i kumulatywnej średniej normalizacji (CMND). YIN redukuje błędy oktawowe i jest referencyjną implementacją stosowaną w systemach czasu rzeczywistego, w tym w UltraStar Deluxe. Złożoność obliczeniowa: O(N log N) z użyciem FFT.

Algorytm pYIN (Mauch i Dixon, 2014)[19] rozszerza YIN o wnioskowanie probabilistyczne za pomocą ukrytego modelu Markova (HMM): zamiast przyjmować jeden kandydat F0, HMM modeluje prawdopodobieństwo wszystkich kandydatów i wygładza

---

[15] G. F. Welch i in., "The Impact of Singing on Brain and Behaviour", Progress in Brain Research, vol. 237, 2018.

[16] E. J. Humphrey, J. P. Bello, Y. LeCun, "Moving Beyond Feature Design: Deep Architectures and Automatic Feature Learning in Music Informatics", Proc. ISMIR, 2012.

[17] J. W. Kim, J. Salamon, P. Li, J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation", Proc. ICASSP, 2018.

[18] A. de Cheveigné, H. Kawahara, "YIN, a fundamental frequency estimator for speech and music", J. Acoust. Soc. Am., vol. 111, nr 4, s. 1917–1930, 2002.

[19] M. Mauch, S. Dixon, "pYIN: A Fundamental Frequency Estimator Using Probabilistic Threshold Distributions", Proc. ICASSP, s. 659–663, 2014.

trajektorię pitch w czasie. Daje to istotnie mniejszy wskaźnik błędów grubych (ang. *gross pitch errors*) i lepszą obsługę pauz. Implementacja dostępna w bibliotece librosa jako `librosa.pyin()`.

CREPE (Kim i in., 2018)[20] (*Convolutional REpresentation for Pitch Estimation*) jest splotową siecią neuronową działającą bezpośrednio na surowej fali PCM (mono, 16 kHz) bez ręcznie projektowanych cech. Sieć o 6 warstwach konwolucyjnych i 360 klasach (co pół centa od 32,7 Hz do 1975,5 Hz) osiąga stan sztuki na benchmarkach MDB-stemSynth i MedleyDB. Decoder Viterbi w wariancie torchcrepe[21] wygładza estymowaną trajektorię, redukując saltacje. Cena: ok. 310 ms na plik WAV 3–5 sekund na GPU GTX 1060.

Salamon i Gómez[22] zaproponowali metodę ekstrakcji melodii z nagrań polifonicznych opartą na właściwościach konturów pitch (ang. *pitch contour characteristics*): filtrowanie kandydatów F0 według ciągłości, zakresu i siły harmonicznych. Podejście to uzasadnia stosowanie etapu separacji źródła wokalnego z miksu przed detekcją F0 jako sposobu na redukcję zakłóceń harmonicznych instrumentów.

2.4. Separacja źródeł audio w kontekście karaoke

Separacja źródeł (ang. *Source Separation*) polega na wydzieleniu składowych (wokal, gitara, perkusja, bas) z nagrania wielościeżkowego. W kontekście karaoke zakłócenia harmoniczne instrumentów mogą utrudniać detekcję F0 głosu.

Z perspektywy metodologicznej stosowane podejścia obejmują: **NMF i maski Wienera** (lata 2000.) — klasyczne metody oparte na faktoryzacji spektrogramu; oraz nowsze architektury głębokiego uczenia, jak **Open-Unmix**[23] (SDR ~6,32 dB) czy **Demucs**[24] (SDR > 8 dB). Metody głębokiego uczenia wymagają GPU i kilku sekund na utwór, co wyklucza zastosowanie w pętli real-time karaoke. Separacja źródeł nie jest objęta testowym zakresem niniejszej pracy i stanowi potencjalny kierunek dalszego rozwoju.

---

[20] J. W. Kim, J. Salamon, P. Li, J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation", Proc. ICASSP, 2018.

[21] J. W. Kim, J. Salamon, P. Li, J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation", Proc. ICASSP, 2018.

[22] J. Salamon, E. Gómez, "Melody Extraction from Polyphonic Music Signals using Pitch Contour Characteristics", IEEE Trans. ASLP, vol. 20, nr 6, s. 1759–1770, 2012.

[23] F.-R. Stöter, S. Uhlich, A. Liutkus, Y. Mitsufuji, "Open-Unmix — A Reference Implementation for Music Source Separation", JOSS, vol. 4, nr 41, 2019.

[24] A. Défossez, "Hybrid Spectrogram and Waveform Source Separation", Proc. ISMIR 2021, arXiv:2111.03600, 2021.

2.5. Analiza istniejących aplikacji karaoke

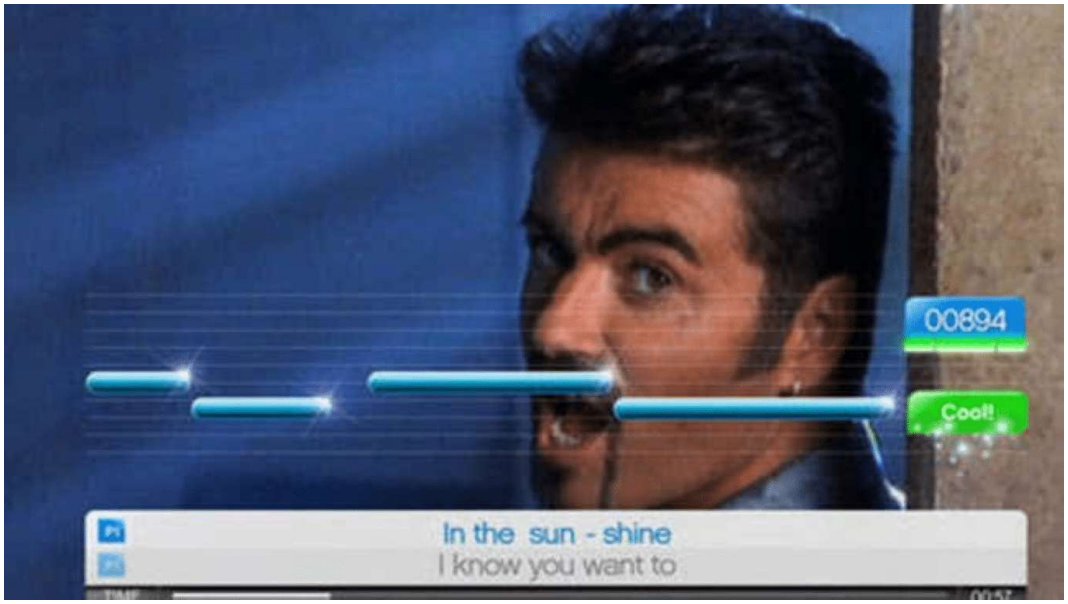
Poniżej przedstawiono analizę siedmiu reprezentatywnych aplikacji karaoke pod kątem wykorzystania metod sztucznej inteligencji, oceny wokalu i modeli biznesowych.

2.5.1. SingStar (Sony Computer Entertainment, 2004)

SingStar zadebiutował w 2004 roku jako ekskluzywna gra wokalna na PlayStation 2. Był pierwszą masową produkcją konsolową, która wprowadza detekcję pitch mikrofonu w czasie rzeczywistym — opierając się na uproszczonej autokorelacji sygnału PCM z mikrofonu USB. Gameplay polega na śpiewaniu wzdłuż widocznej linii nut; kursor śpiewaka wędruje w górę i w dół zależnie od wykrytej wysokości, a punktacja rośnie za każdy beat spędzony w pasie tolerancji (±2 semitony).

System ten, choć prymitywny z perspektywy współczesnej, ustanowił wzorzec interfejsu karaoke z wizualną reprezentacją nut i oceną ilościową (rys. 2.1), który stał się bezpośrednią inspiracją dla UltraStar Deluxe i AudioVerse. Tryb dwuosobowy pozwalał rywalizować w czasie rzeczywistym.

Ograniczenia: wyłącznie platforma PlayStation, brak modeli AI, prosta detekcja pitch, zamknięta biblioteka piosenek.



Rysunek 2.1 — *Interfejs gry SingStar (PlayStation 2): pasek nut z kursorem śpiewaka i bieżącą punktacją. Stanowi wzorzec interfejsu karaoke z oceną pitch. Źródło: materiały*

prasowe Sony Computer Entertainment Europe (screeny dostępne w serwisach gamepress.gg i gaming.wikia.com).[25]

2.5.2. UltraStar Deluxe

UltraStar Deluxe (USDX) to otwarty, wieloplatformowy klon SingStar rozwijany od 2007 roku jako projekt społecznościowy. Kluczowy wkład techiczny:

- **Format UltraStar TXT** — otwarty tekstowy format pliku definiujący nuty, sylaby, tempa i metadane. Plik zawiera nagłówek (#TITLE, #ARTIST, #BPM, #GAP, #MP3) i sekcję nut (`:`/`*`/`F`/`-`), opisaną szczegółowo w sekcji 2.12.
- **Pitch tracking YIN** — implementacja algorytmu YIN w C++ do detekcji F0 mikrofonu w czasie rzeczywistym.
- **Octave folding** — algorytm składania oktaw (±6 semitonów), przeniesiony dosłownie do funkcji `hzToUltrastarPitch()` w AudioVerse.

Ogromna biblioteka piosenek stworzona przez społeczność (>10 000 plików TXT) czyni USDX platformą referencyjną dla projektów open-source karaoke (rys. 2.2).

Ograniczenia: brak metod ML/AI, deskopowy charakter, brak komponentu webowego, estymacja pitch YIN ustępuje współczesnym algorytmom dokładnością. Dodatkową barierą wejścia — zaobserwowaną przez autora w praktyce — jest brak centralnego repozytorium piosenek i niewystandaryzowany sposób dystrybucji plików TXT: użytkownik musi samodzielnie odnaleźć kompatybilny plik audio i dopasować go do pobranego pliku TXT, co potrafi zająć więcej czasu niż sama rozgrywka.

[25] Źródło ilustracji: Appleby Games, https://applebygames.co.nz/shop/singstar-chart-hits-ps2-rare/?srsltid=AfmBOo-rulWWGaBsYfgSYAJTiSrch_FBI9XLrE3kfpq1_c0XtbBLS8tzo, dostęp: 2026-02-20.

Rysunek 2.2 — *Ekran rozgrywki UltraStar Deluxe: timeline nut z sylabami, kursor pitch gracza i bieżąca punktacja. Interfejs wzorowany na SingStar, open-source z pełną obsługą formatu UltraStar TXT. Źródło: materiały z oficjalnej strony projektu UltraStar Deluxe (usdx.eu).*[26]

2.5.3. Smule

Smule to mobilna platforma społecznościowa karaoke (iOS/Android, 2012–). Kluczową propozycją wartości jest tryb duetowy (*Join*): użytkownik nagrywa swoją partię na istniejącym nagraniu innego użytkownika, tworząc wirtualny duet. Platforma oferuje efekty głosowe, filtry AR i kreację wideo.

Ocena wokalu opiera się wyłącznie na metrykach społecznościowych (lajki, komentarze) — brak automatycznej analizy intonacji (rys. 2.3). Model biznesowy (freemium, subskrypcja ok. 7,99 USD/miesiąc) nie obejmuje funkcji AI do coachingu.

[26] Źródło ilustracji: Wikipedia: UltraStar, https://en.wikipedia.org/wiki/UltraStar, dostęp: 2026-02-18.

Rysunek 2.3 — *Interfejs Smule w trybie duetowym (Join): podświetlony tekst piosenki, warstwa wideo z kamery, brak wskaźnika pitch. Ocena opiera się wyłącznie na metrykach społecznościowych (lajki, komentarze). Źródło: materiały prasowe Smule (smule.com).*[27]

2.5.4. SingSnap

SingSnap (Kanada, 2006–) to webowa platforma do nagrywania i udostępniania wykonań karaoke. Brak mechanizmu oceny wokalu — użytkownik rejestruje wykonanie i publikuje je społeczności. Platforma koncentruje się na budowaniu portfolio wokalnego i grywalizacji społecznej (rankingi, odznaki).

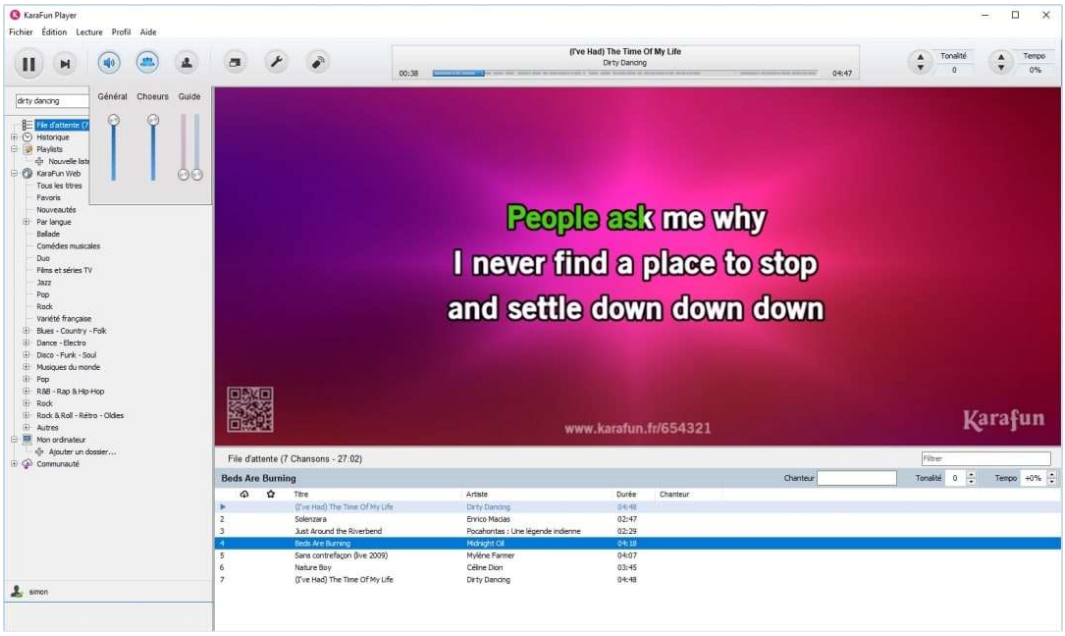
[27] Źródło ilustracji: Smule, https://www.smule.com/, dostęp: 2026-02-18.

2.5.5. Yokee

Yokee (iOS/Android) korzysta z zasobów YouTube jako źródła ścieżek. Aplikacja nakłada tekst i kolorowe tło na odtwarzany teledysk. Brak jakiegokolwiek mechanizmu oceny śpiewu lub detekcji pitch. Służy wyłącznie jako nakładka tekstu.

2.5.6. KaraFun

KaraFun to profesjonalne rozwiązanie dla prowadzących eventy karaoke (Windows/Mac/iOS/Android). Oferuje licencjonowaną bazę ponad 41 000 piosenek z prawdziwymi naprawionymi ścieżkami wokalnymi i instrumentalnymi. Tryb offline, tryb DJ (kolejkowanie piosenek przez publiczność). Brak AI — ocena wokalu nieobecna (rys. 2.4); produkt skierowany do profesjonalistów prowadzących imprezy.



Rysunek 2.4 — *Interfejs KaraFun Desktop: licencjonowana baza piosenek, tryb DJ z kolejkowaniem przez publiczność i synchronizowany tekst. Brak jakiejkolwiek analizy wokalu lub oceny intonacji. Źródło: materiały z oficjalnej strony KaraFun (karafun.com).*[28]

[28] Źródło ilustracji: Karafun, https://www.karafun.com/karaokeplayer/, dostęp: 2026-02-18.

2.5.7. Karaoke One

Karaoke One oferuje edytor wideo karaoke (tworzenie własnych plików .cdg), system oceny przez społeczność i bazę ponad 8 000 piosenek. Podobnie jak SingSnap, kładzie nacisk na treści generowane przez użytkowników (UGC) bez automatycznej analizy wokalu.

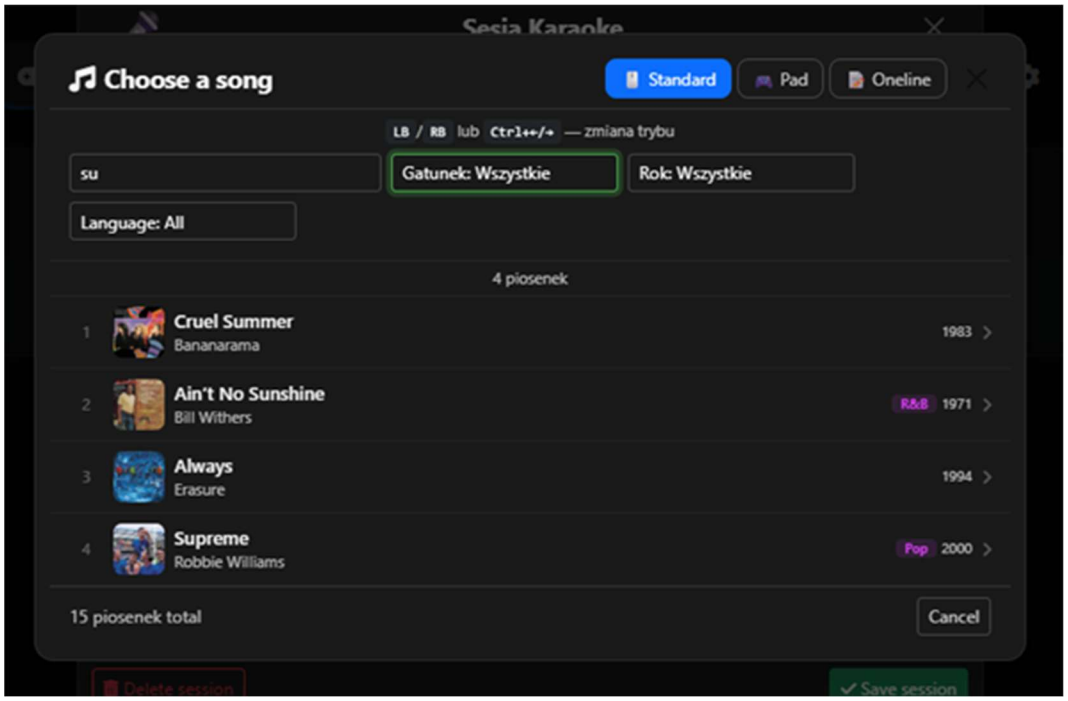
2.6. Wnioski z analizy — luka w istniejących rozwiązaniach

Tabela 2.1 zestawia analizowane systemy według kluczowych kryteriów.

Tabela 2.1. Porównanie aplikacji karaoke

Dane na podstawie oficjalnych stron produktów, recenzji branżowych (2024) oraz własnej analizy dostępnych funkcji. Kolumna „Pitch tracking" oznacza automatyczną detekcję F0 z mikrofonu — bez uwzględnienia ewentualnych funkcji AI oferowanych w nowszych wersjach po dacie analizy.

Aplikacja	Platforma	Pitch tracking	Ocena AI	Open source	Model biznesowy
SingStar	PS2/PS3/PS4	Autokorelacja	Nie	Nie	Płatna gra
UltraStar Deluxe	Win/Mac/Linux	YIN (C++)	Nie	Tak (GPL)	Darmowy
Smule	iOS/Android	Brak	Nie	Nie	Freemium
SingSnap	Web	Brak	Nie	Nie	Freemium
Yokee	iOS/Android	Brak	Nie	Nie	Reklamy
KaraFun	Win/Mac/Mob.	Brak	Nie	Nie	Subskrypcja
Karaoke One	Web/Android	Brak	Nie	Nie	Freemium



Rysunek 2.5 — *Przeglądarka piosenek AudioVerse z filtrowaniem po tytule, artyście, języku i gatunku. Widoczna lista kafelków piosenek z metadanymi, paginacja i skeleton loadery podczas ładowania. Źródło: własne, zrzut ekranu aplikacji AudioVerse, widok KaraokeSongBrowserPage (*`audioverse-react/src/pages/party/KaraokeSongBrowserPage.tsx`*).*[29]

Zestawienie analizowanych systemów przedstawiono w tabeli 2.1, a przykładowy widok przeglądarki piosenek AudioVerse na rysunku 2.5.

Z analizy wynika, że żadna z komercyjnych platform karaoke nie implementuje zaawansowanych metod AI do automatycznej oceny wokalu (kolumna *Ocena AI: Nie* dla wszystkich komercyjnych aplikacji). Dostępne open-source systemy (UltraStar) stosują algorytmy klasyczne bez uczenia maszynowego. Luka ta polega na braku webowej platformy open-source, która:

- implementuje detekcję F0 opartą na deep learning (CREPE),
- stosuje separację źródeł Demucs jako krok preprocessingowy,
- ocenia śpiew metodą DTW w czasie rzeczywistym,

[29] Źródło ilustracji: Źródło własne — zrzut ekranu aplikacji AudioVerse, audioverse-react/src/pages/party/Karaoke-SongBrowserPage.tsx, dostęp: 2026-02-20.

- dostarcza mierzalnej analizy komparatywnej algorytmów w dedykowanym module laboratoryjnym,
- działa w przeglądarce bez instalacji aplikacji klienckiej.

AudioVerse realizuje wszystkie pięć wymienionych punktów: detekcję F0 modelem CREPE, separację źródeł Demucs jako krok preprocessingowy, ocenę DTW, moduł laboratoryjny z dziewięcioma endpointami pomiarowymi oraz wdrożenie webowe bez instalacji aplikacji klienckiej.

2.7. Cyfrowe przetwarzanie sygnałów audio — podstawy

Sygnał dźwiękowy to ciągła funkcja ciśnienia akustycznego $p(t)$ w dziedzinie czasu. W systemach cyfrowych podlega on procesowi ADC: próbkowaniu z częstotliwością f_s (standardowo 16 000 Hz dla analizy mowy; 44 100 Hz dla muzyki) i kwantyzacji do reprezentacji stałoprzecinkowej (zazwyczaj 16 bitów PCM)[30].

Transformata Fouriera pozwala przejść do dziedziny częstotliwości. Dyskretna transformata Fouriera (DFT) próbki sygnału $x[n]$ długości N:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi k\ /N}, k = 0, \dots, N-1$$

W praktyce stosuje się FFT (algorytm szybkiej DFT) o złożoności $O(N\log N)$.

Spektrogram STFT (Short-Time Fourier Transform) obliczany jest przez nakładanie okna analizującego (Hanninga lub Blackmana) na nakładające się fragmenty sygnału i obliczanie FFT każdego z nich. Oś y wyraża częstotliwość, oś x czas, jasność piksela — amplitudę współczynnika Fouriera[31].

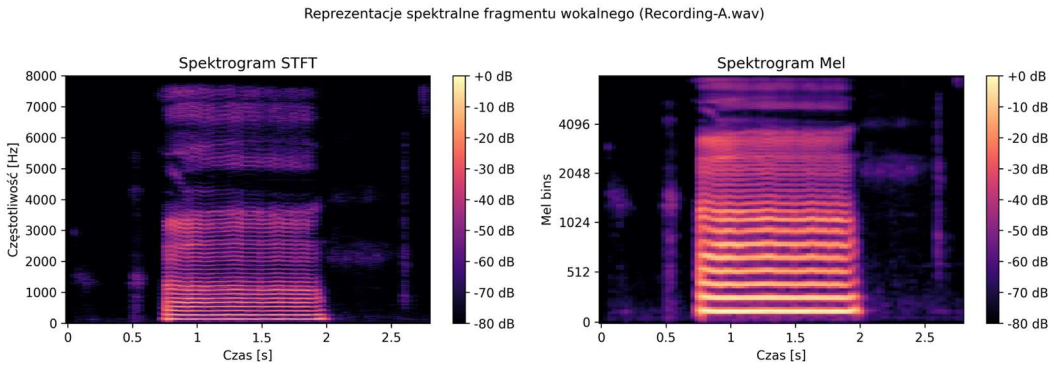
Skala melowa odwzorowuje liniową skalę częstotliwości na percepcyjną skalę melową (bardziej gęstą poniżej 1 kHz i mniej gęstą powyżej):

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

Spektrogram Mel uzyskuje się przez przemnożenie macierzy STFT przez filtr bankowy w skali melowej (standardowo 128 filtrów) — jest to wejście do CREPE i kolejnych sieci DNN (rys. 2.6).

---

[30] T. Zieliński, Cyfrowe przetwarzanie sygnałów. Od teorii do zastosowań, Wydawnictwo Komunikacji i Łączności, 2009.
[31] M. Müller, Fundamentals of Music Processing: Using Python and Jupyter Notebooks, wyd. 2, Springer, 2021.



Rysunek 2.6 — *Spektrogram STFT (lewy) i spektrogram Mel (prawy) 3-sekundowego fragmentu wokalnego. Oś Y lewego: częstotliwość [Hz]; oś Y prawego: skala melowa (128 filtrów). Widoczne harmoniczne głosu jako jaśniejsze poziome linie. Źródło: własne, wygenerowany skryptem Python (librosa + matplotlib) na przykładowym nagraniu wokalnym (mono, 16 kHz).*[32]

2.8. Ekstrakcja cech akustycznych

Spośród cech akustycznych stosowanych w analizie wokalu kluczowe są[33]:

MFCC (Mel-Frequency Cepstral Coefficients) — logarytm energii wyjść filtrów melowych, transformowany przez DCT-II. Pierwsze 13–20 współczynników koduje barwę głosu. Stosowane w klasyfikacji tonu głosu, ale nieużywane bezpośrednio do detekcji F0 (zob. sekcja 2.9).

Chromagram — projekcja spektrogramu na 12 klas wysokości (C, C#, D, ...) poprzez sumowanie energii bin FFT odpowiadających każdemu pitch class. Umożliwia reprezentację harmoniczną niezależną od oktawy.

Energia sygnału (RMS) — używana przez mikroserwis `audio_vad` do detekcji aktywności głosu (VAD, ang. *Voice Activity Detection*): ramki z RMS < progu klasyfikowane są jako pauza, co zapobiega zliczaniu fałszywych trafień.

---

[32] Źródło ilustracji: Źródło własne — spektrogram wygenerowany biblioteką librosa + matplotlib, audioverse-react/src/pages/admin/lab/LabPage.tsx, dostęp: 2026-02-20.

[33] A. Lerch, An Introduction to Audio Content Analysis: Applications in Signal Processing and Music Informatics, Wiley, 2012.

2.9. Detekcja wysokości dźwięku (pitch detection)

Częstotliwość podstawowa F_0 to najniższa składowa harmoniczna sygnału quasi-periodycznego wydawanego przez struny głosowe. Dla dobrego tenora $F_0 \in [130, 523]$ Hz, dla sopranu [261, 1046] Hz.

2.9.1. Algorytm YIN i jego ograniczenia

YIN[34] wyznacza F_0 przez minimalizację kumulatywnej średniej funkcji różnicowej (CMND):

1 & \tau = 0 \\ \dfrac{d(\tau)}{\frac{1}{\tau}\sum_{j=1}^{\tau} d(j)} & \tau > 0 \end{cases}$$

gdzie $d(\tau) = \sum_{j=1}^{W}(x_j - x_{j+\tau})^2$ to kwadratowa funkcja różnicowa, W — rozmiar okna.

Pierwsza wartość τ spełniająca $d'(\tau) < \theta$ (typowo $\theta = 0.1$) daje $F_0 = f_s/\tau$. YIN działa w $O(N\log N)$ z FFT lub $O(N^2)$ w wersji naiwnej. Słabe strony: wrażliwość na wielodźwięki, błędy oktawowe przy barwach z niską energią fundamentu.

2.9.2. Probabilistyczny pYIN

pYIN[35] (ICASSP 2014) wydaje wielokandydatową listę wartości F_0 z wagami prawdopodobieństwa. Ukryty model Markova (HMM) ze stanami *voiced* i *unvoiced* wygładza trajektorię przez algorytm Viterbiego. Implementacja w librosa:

```
f0, voiced_flag, voiced_probs = librosa.pyin(
    y, fmin=librosa.note_to_hz("C2"),
    fmax=librosa.note_to_hz("C7"),
    sr=sr, frame_length=2048
)
```

Rezultat: tablica F0 (NaN w chwilach *unvoiced*), flaga *voiced*, pewność ramki. Latencja na CPU: ~78 ms dla pliku 3–5 s (16 kHz, mono).

---

[34] A. de Cheveigné, H. Kawahara, "YIN, a fundamental frequency estimator for speech and music", J. Acoust. Soc. Am., vol. 111, nr 4, s. 1917–1930, 2002.

[35] M. Mauch, S. Dixon, "pYIN: A Fundamental Frequency Estimator Using Probabilistic Threshold Distributions", Proc. ICASSP, s. 659–663, 2014.

2.9.3. CREPE — sieć konwolucyjna do detekcji F0

CREPE[36] (ICASSP 2018) to 6-warstwowa 1D-CNN operująca bezpośrednio na fragmencie surowej fali 1024 próbek (64 ms @ 16 kHz). Sieć zwraca wektor 360 logitów odpowiadających binom co 20 centów (od C1 = 32,7 Hz do B6 = 1975,5 Hz). Pytanie o F0 redukuje się do klasyfikacji wieloklasowej:

$$\hat{f} = \text{argmax}_k \, p_k, p_k = \text{softmax}(\mathbf{z})_k$$

Wariant z dekoderem Viterbi (torchcrepe) modeluje tranzycje między stanami HMM i wygładza trajektorię, eliminując saltacje. Czas przetwarzania: ~310 ms (GPU GTX 1060) dla pliku 3–5 s. Przewyższa pYIN na benchmarkach MDB-stemSynth (RPA @ 50 centów: 97,4% vs 95,8%).

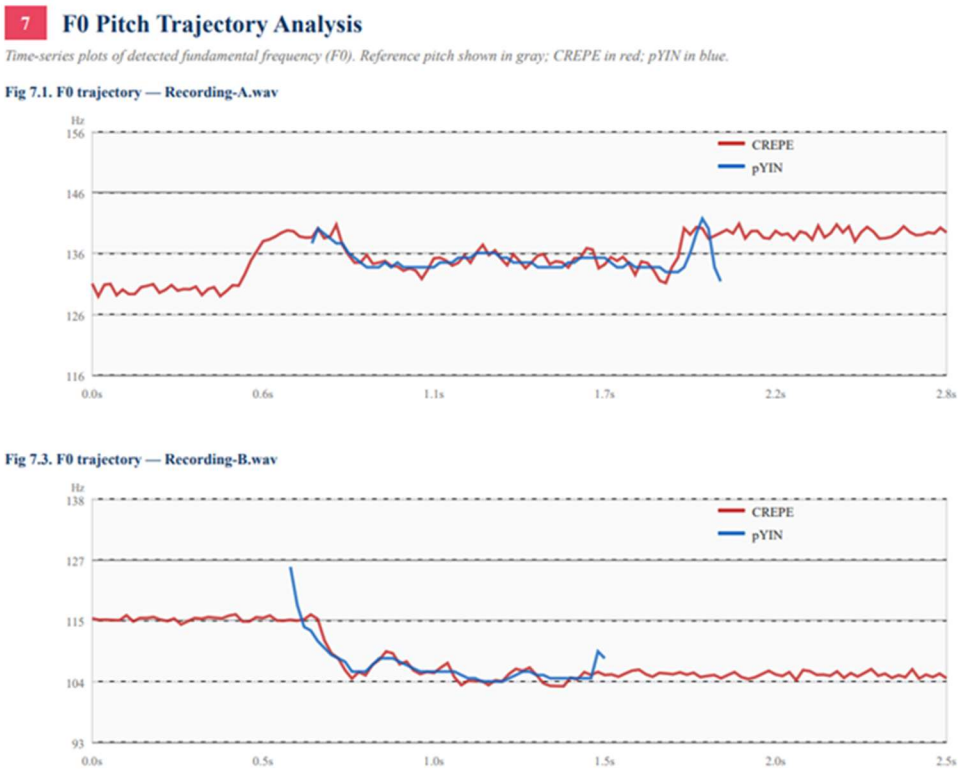
2.9.4. Porównanie algorytmów

Zestawienie porównawcze głównych cech omawianych algorytmów przedstawia tabela 2.2, a wizualną porównanie trajektorii F0 — rysunek 2.7.

Tabela 2.2. Porównanie algorytmów detekcji F0

Cecha	YIN	pYIN	CREPE
Typ	Klasyczny DSP	Probab. DSP	Deep learning
Platforma	CPU	CPU	GPU (fallback CPU)
Latencja (3–5 s)	~15 ms	~78 ms	~310 ms
RPA @ 50c (czysty wokal)	~91%	~95,8%	~97,4%
Odporność na szum	Niska	Średnia	Wysoka
Wymagania GPU	Nie	Nie	Tak (zalecane)

[36] J. W. Kim, J. Salamon, P. Li, J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation", Proc. ICASSP, 2018.



Rysunek 2.7 — *Porównanie trajektorii F0 wykrytych przez pYIN i CREPE dla tego samego 5-sekundowego pliku wokalnego. Oś Y: częstotliwość [Hz], oś X: czas [s]. Linia niebieska — pYIN, czerwona — CREPE, szara — referencja. Źródło: własne, wygenerowany przez endpoint* `POST /api/karaoke/lab/pitch/compare`*, wizualizacja z raportu PDF zwracanego przez* `POST /api/karaoke/lab/report/pdf`.[37]

2.10. Dopasowanie czasowe — Dynamic Time Warping (DTW)

Dynamic Time Warping (DTW)[38] to algorytm dopasowania dwóch sekwencji czasowych o różnej długości lub zmiennym tempie, poprzez nieliniowe rozciąganie osi czasu — każda ramka pierwszej sekwencji może być dopasowana do jednej lub kilku ramek drugiej, bez konieczności synchronizacji 1:1. Algorytm wywodzi się z rozpoznawania mowy (Sakoe

[37] Źródło ilustracji: Źródło własne — wykres z endpointa POST /api/karaoke/lab/pitch/compare, audioverse-react/src/pages/admin/lab/LabPage.tsx, dostęp: 2026-02-20.

[38] H. Sakoe, S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition", IEEE Trans. ASSP, vol. 26, nr 1, s. 43–49, 1978.

i Chiba, 1978) i jest stosowany wszędzie tam, gdzie dwie sekwencje mierzą to samo jawisko, lecz przebiegają z różnym tempem lub z lokalnymi opóźnieniami.

W systemie AudioVerse DTW stanowi rdzeń mikroserwisu `sing_score`: porównuje wykrytą trajektorię F0 śpiewaka z referencyjną sekwencją nut z pliku UltraStar, tolerując naturalne wahania tempa wykonania. Wynik DTW normalizowany jest do skali 0–100 i zapisywany w polu `Score` encji `KaraokeSinging`.

Formalnie DTW minimalizuje odległość akumulowaną:

$$D(i,j) = d(q_i, r_j) + min[D(i-1,j), D(i,j-1), D(i-1,j-1)]$$

gdzie q_i i r_j to kolejne ramki F0 śpiewaka i odniesienia, $d(q_i, r_j) = |q_i - r_j|$ (lub odległość w centach).

Wynik DTW $D(N, M)$ jest unormalizowaną sumą kosztów dopasowania ścieżki — mniejsza wartość oznacza lepsze wykonanie. Mikroserwis `sing_score` normalizuje wynik do skali 0–100. Biblioteka dtaidistance (Cython+ OpenMP) daje czas DTW < 20 ms dla trajektorii długości 300 ramek.

2.12. Format UltraStar TXT

Format UltraStar TXT jest de facto standardem dla open-source karaoke. Plik składa się z nagłówka i sekcji nut.

Nagłówek (przykład):

```
#TITLE:Bohemian Rhapsody
#ARTIST:Queen
#MP3:bohemian_rhapsody.mp3
#BPM:148.00
#GAP:3500
#LANGUAGE:English
```

Pole `#GAP` to opóźnienie startu nut względem początku pliku audio (ms). Pole `#BPM` wyraża tempo piosenek jako czwarte beaty na minutę (w UltraStar WorldParty *BPM = rzeczywiste BPM × 4*; obsługiwane są oba warianty przez `extractBpmFromNotes()`).

Linia nuty ma format: `typ beat duracja pitch sylaba`, np.:

```
: 32 4 5 Mama
: 36 4 5 ,
* 40 4 7 just
: 44 4 5 killed
F 200 4 0 ~
- 260
```

Typy: `:` — zwykła nuta, `*` — złota nuta, `F` — freestyle (bez oceny), `-` — linia przerwy (pauza dla oddychania), `#` — rap.

Przeliczanie beat → sekundy:

$$t[\mathrm{s}] = \frac{\mathrm{beat} \times 15}{\mathrm{BPM}} + \frac{\mathrm{GAP}}{1000}$$

Wzór pochodzi z `URecord.pas` projektu UltraStar WorldParty i jest implementowany w `karaokeNoteParsing.ts`:

```
export function parseNotes(
    notes: string[], bpmOverride?: number
): KaraokeNoteData[][] {
    const bpm = bpmOverride ?? extractBpmFromNotes(notes) ?? 120;
    // ...
    const startTime = (beat * 15) / bpm + gap / 1000;
    const endTime   = ((beat + duration) * 15) / bpm + gap / 1000;
}
```

Funkcja `hzToUltrastarPitch()` odwzorowuje F0 w Hz na pitch class (0–11) z uwzględnieniem oktawowego zawijania (ang. *octave folding*):

```
function hzToUltrastarPitch(hz: number, targetNotePitch: number): number {
    const midi  = Math.round(69 + 12 * Math.log2(hz / 440));
    let detected = ((midi % 12) + 12) % 12;      // pitch class 0–11
    // octave folding — dopasowanie najblizszej oktawy:
    while (detected - targetNotePitch >  6) detected -= 12;
    while (detected - targetNotePitch < -6) detected += 12;
    return detected;
}
```

Algorytm octave folding pochodzi z oryginalnego kodu UltraStar i zapobiega błędom oktawowym przy porównywaniu z nutą referencyjną.

Rozdział 3 — Projekt i architektura systemu

3.1. Wymagania funkcjonalne i niefunkcjonalne

System AudioVerse był projektowany na podstawie zestawu wymagań zebranych w dwóch kategoriach (tab. 3.1).

Tabela 3.1. Wymagania funkcjonalne (WF)

ID	Treść wymagania
WF01	Użytkownik może się zarejestrować i zalogować (Identity Server, JWT)
WF02	Biblioteka piosenek: przeglądanie, filtrowanie po tytule/artyście/gatunku
WF03	Import piosenki w formacie UltraStar TXT wraz z plikiem audio

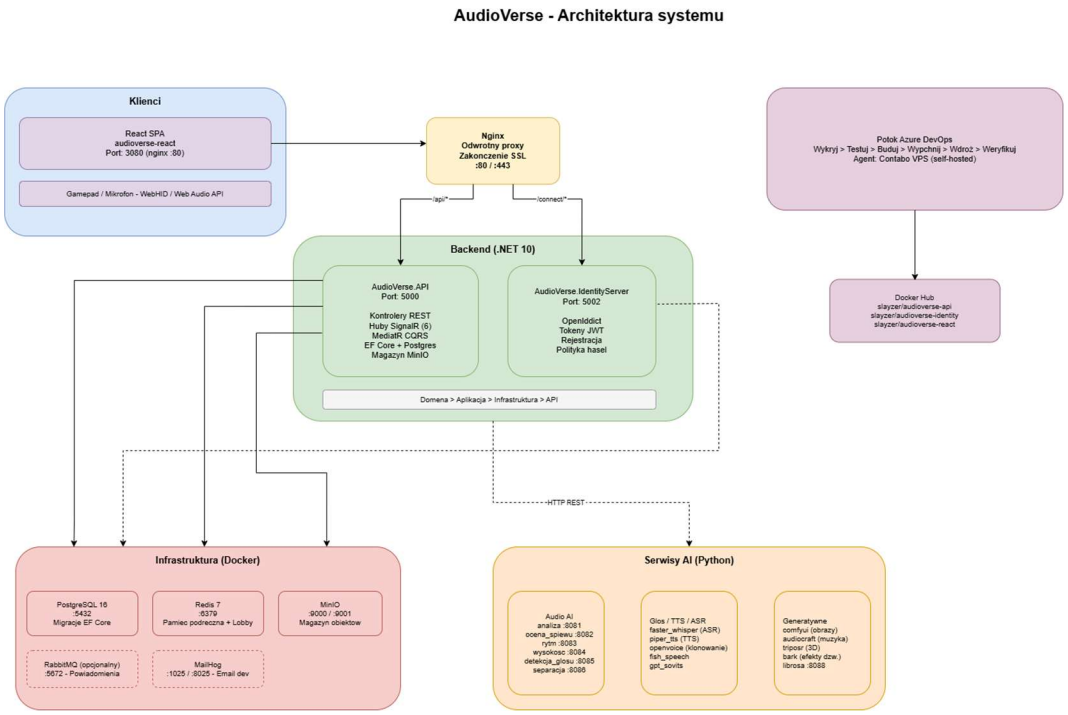
WF04	Tworzenie sesji (party) karaoke z kodem dołączenia
WF05	Tryb gry: wizualizacja nut, detekcja pitch w czasie rzeczywistym
WF06	Ocena śpiewu per nuta (perfect/good/miss) i Score 0–10 000 pkt
WF07	Wyświetlenie rankingu po zakończonej rundzie
WF08	Moduł laboratoryjny: pomiary algorytmów detekcji F0 i generowanie PDF

Wymagania niefunkcjonalne (WNF):

- WNF01: Latencja detekcji pitch (pYIN real-time) < 150 ms od wysłania PCM.
- WNF02: System działa w przeglądarce bez konieczności instalowania wtyczek.
- WNF03: Aplikacja jest wdrażana jedną komendą (`docker compose up`).
- WNF04: Użytkownik może korzystać z systemu na ekranach ≥ 1280 × 720 px.
- WNF05: Raporty PDF z modułu laboratoryjnego generowane w < 5 s.

3.2. Architektura systemu

System AudioVerse zbudowany jest w oparciu o architekturę mikroserwisową złożoną z czterech warstw logicznych komunikujących się przez REST, WebSocket i SignalR. Pełna architektura systemu z zależnościami między warstwami przedstawiona jest na rysunku 3.1.



Rysunek 3.1 — *Schemat komunikacji między warstwami systemu AudioVerse: przeglądarka → ASP.NET API → mikroserwisy AI (FastAPI) → PostgreSQL / MinIO. Dla każdego mikroserwisu podano protokół (HTTP/WebSocket) i średnią latencję. Źródło: własne, diagram dostępny w pliku* `AudioVerse.API/Docs/diagrams/01-system-architecture.drawio`.[39]

Frontend przesyła strumień PCM przez surowe WebSocket do mikroserwisu `audio_pitch`. Wyniki F0 wracają do frontendu, gdzie silnik karaoke (`karaokeTimelineRenderer.ts`) nanosi kursor pitch na wizualizację nut. Backend `.NET` pośredniczy w synchronizacji stanu sesji przez SignalR i zapisuje wyniki gry do PostgreSQL.

3.3. Model danych

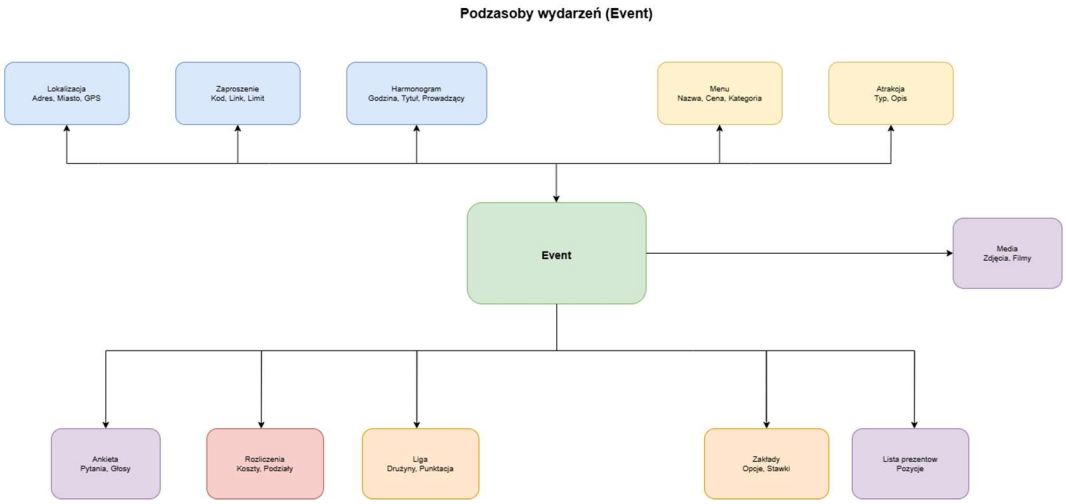
Moduł karaoke oparty jest na siedmiu encjach głównych (tab. 3.2, rys. 3.2):

Tabela 3.2. Encje domeny karaoke

Encja	Kluczowe pola

[39] Źródło ilustracji: Źródło własne — diagram architektury systemu AudioVerse, audioverse-dotnet/AudioVerse.API/Docs/diagrams/01-system-architecture.drawio, dostęp: 2026-02-20.

KaraokeSong	Id, Title, Artist, Language, Genre, CoverImagePath
KaraokeSongFile	Id, SongId, S3Key, Format (UltraStar/MP3), BPM, GAP
KaraokeSession	Id, Code (join code), HostId, Status, CreatedAt
KaraokeSessionRound	Id, SessionId, SongFileId, Status, StartedAt, EndedAt
KaraokeSessionRoundPlayer	Id, RoundId, PlayerId, Slot (1–4)
KaraokeSinging	Id, RoundId, RoundPartId, PlayerId, **Score, Hits, Misses, Good, Perfect, Combo**
KaraokeSingingRecording	Id, SingingId, S3Key, DurationMs



Rysunek 3.2 — *Diagram encji modelu danych modułu karaoke (ER). Widoczne relacje między KaraokeSession, KaraokeSessionRound, KaraokeSessionRoundPlayer i KaraokeSinging. Źródło: własne, wygenerowany diagram dostępny w pliku*

`AudioVerse.API/Docs/diagrams/auto-data-model.drawio`, *wygenerowany narzędziem* `AudioVerse.DiagramGenerator`.[40]

Pole `Score` w `KaraokeSinging` przechowuje skumulowane punkty gracza (0–10 000). `Hits`, `Misses`, `Good`, `Perfect` to liczniki wynikające z oceny per nuta. `Combo` to maksymalna seria kolejnych trafionych nut uzyskana w trakcie rundy.

3.4. Stos technologiczny

System AudioVerse został zbudowany w oparciu o technologie dedykowane poszczególnym warstwom: backend w ASP.NET Core/.NET 10 (C#), mikroserwisy AI w Python 3.11 z FastAPI, frontend w React 18 + TypeScript. Pełne zestawienie wyborów technologicznych wraz z uzasadnieniem zawiera tabela 3.3. Szczegółowy schemat komunikacji między warstwami i zależności serwisów przedstawiono w podrozdziale 3.2 (rys. 3.1); implementacja komunikacji omówiona jest szczegółowo w rozdziale 4.

Tabela 3.3. Stos technologiczny AudioVerse

Warstwa	Technologia	Uzasadnienie wyboru
Backend API	ASP.NET Core 10, C# 13	Typowanie, CQRS, wbudowany SignalR
ORM	Entity Framework Core 10	Code-first migrations, LINQ
CQRS/Bus	MediatR 12	Separacja komend i zapytań
Real-time	SignalR (WebSocket transport)	Automatyczny fallback, .NET SDK
Frontend	React 18 + TypeScript 5.3	Ekosystem, typowanie, React Query
Rendering gry	Canvas API (2D context)	Brak zewnętrznych zależności, 60fps

[40] Źródło ilustracji: Źródło własne — diagram ER modelu danych karaoke, audioverse-dotnet/AudioVerse.API/Docs/diagrams/auto-data-model.drawio, dostęp: 2026-02-20.

Mikroserwisy AI	Python 3.11, FastAPI 0.135	Ekosystem ML (librosa, torch)
Pitch (fast)	librosa pYIN	CPU-only, niska latencja
Pitch (accurate)	torchcrepe (CREPE + Viterbi)	GPU, wysoka dokładność
Baza danych	PostgreSQL 16	ACID, JSONB, dojrzałość
Pliki audio/video	MinIO (S3-compatible)	Nowoczesne rozwiązanie chmurowe
Cache / pub-sub	Redis Stack	Sessions, kolejki zdarzeń
Raportowanie PDF	QuestPDF 2024.10	Code-first PDF, licencja MIT
Monitoring	Prometheus + OpenTelemetry	Metryki latencji mikroserwisów
Wdrożenie	Docker Compose	Reprodukowalne środowisko

3.5. Komunikacja w czasie rzeczywistym — ogólna koncepcja

W sesji karaoke wymagana jest dwukierunkowa, niskolatencyjna komunikacja między przeglądarką a serwerem. W systemie AudioVerse zastosowano dwa kanały: protokół SignalR do synchronizacji stanu sesji oraz surowe WebSocket do strumieniowania PCM z mikrofonu. Dzięki temu możliwe jest działanie detekcji pitch w czasie rzeczywistym z latencją poniżej 150 ms. Szczegółowa implementacja obu kanałów opisana jest w rozdziale 4.

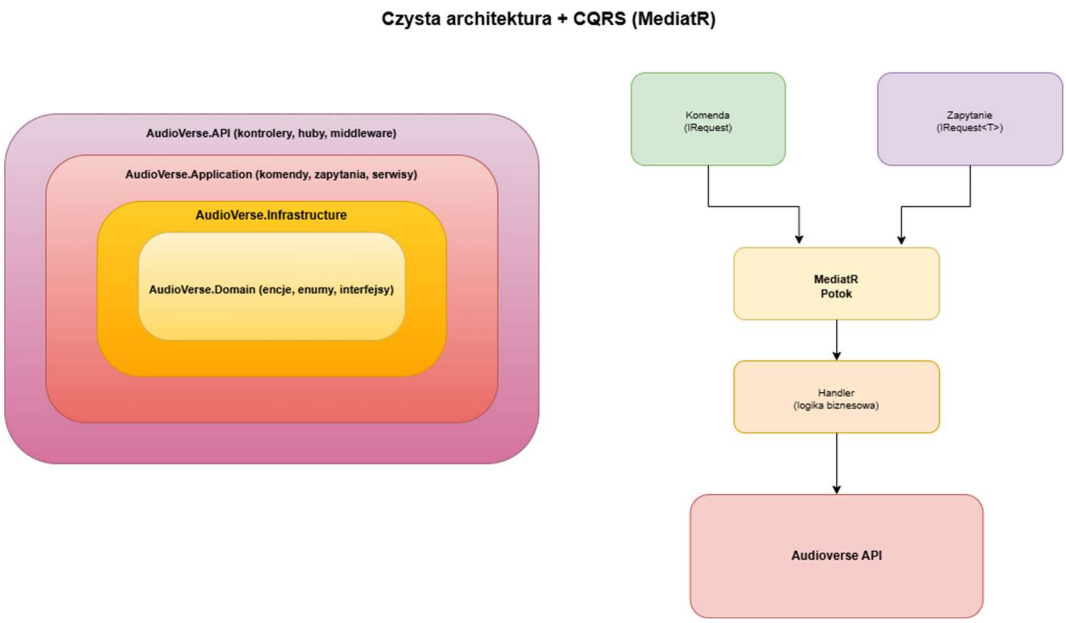
Rozdział 4 — Implementacja

4.1. Backend — AudioVerse.API (.NET 10)

Backend zorganizowany jest według wzorca CQRS zaimplementowanego przez bibliotekę MediatR 12 (rys. 4.1). Każda operacja klienta transluje się na komendę (`Command`) lub zapytanie (`Query`), obsługiwane przez dedykowane `Handler` w warstwie `AudioVerse.Application`. Kontrolery są cienkie — zawierają jedynie akcję HTTP i delegację do mediatora:

```
[HttpPost("singing")]
public async Task<IActionResult> SaveSinging(
    [FromBody] SaveKaraokeSingingCommand cmd,
```

```
    CancellationToken ct)
    => Ok(await _mediator.Send(cmd, ct));
```



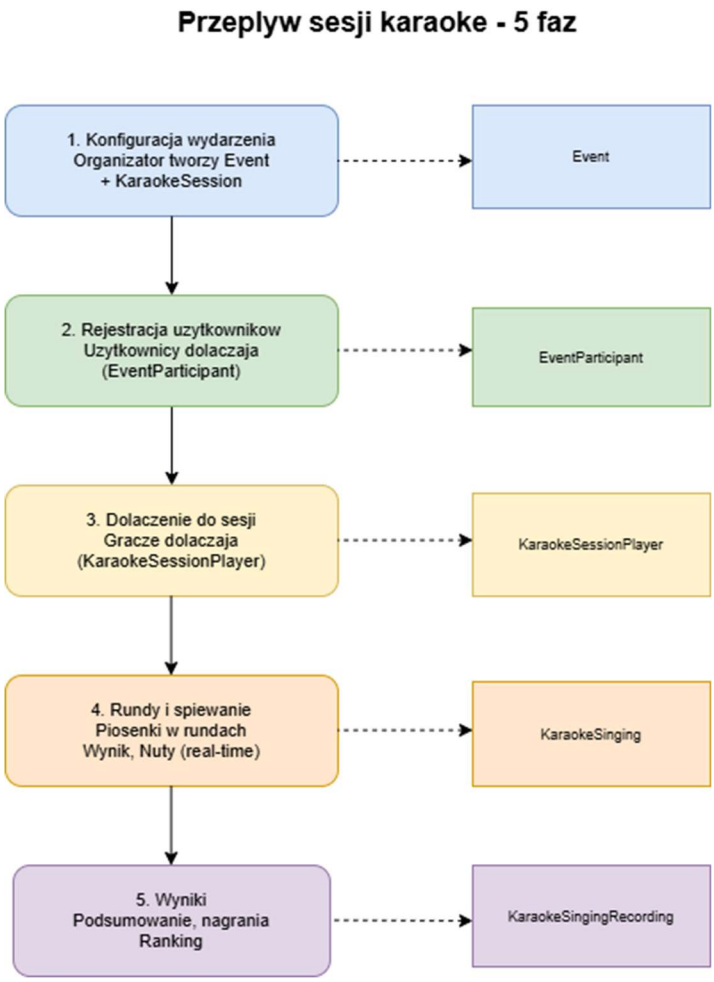
Rysunek 4.1 — *Diagram architektury CQRS w AudioVerse: Controller → MediatR → Handler → Repository → DbContext → PostgreSQL. Widoczne przykłady: SaveKaraokeSingingCommand → SaveKaraokeSingingHandler (zapis wyników) i GetRankingsQuery → GetRankingsHandler (odczyt rankingów). Źródło: własne, diagram dostępny w pliku* `AudioVerse.API/Docs/diagrams/08-cqrs-architecture.drawio`.[41]

4.1.1. Zarządzanie sesjami i rundami

`KaraokeController` (trasa `/api/karaoke`) udostępnia operacje (rys. 4.2): tworzenie sesji (`CreateSession`), dołączenie przez kod (`JoinByCode`), tworzenie i konfigurowanie rundy (`CreateRound`, `AssignPlayers`), zapis wyników śpiewania (`SaveSinging`), pobieranie rankingów.

Stan sesji synchronizowany jest w czasie rzeczywistym przez `KaraokeHub` (SignalR) — po zakończeniu rundy serwer emituje zdarzenie `RoundFinished` do klienta gracza.

[41] Źródło ilustracji: Źródło własne — diagram architektury CQRS z MediatR, audioverse-dotnet/AudioVerse.API/Docs/diagrams/08-cqrs-architecture.drawio, dostęp: 2026-02-20.



Rysunek 4.2 — *Diagram przepływu sesji karaoke: CreateSession → kod dołączenia → JoinByCode → CreateRound → AssignPlayers → strumieniowanie PCM → detekcja pitch → zapis wyników → RoundFinished → ekran rankingu. Źródło: własne, diagram dostępny w pliku* `AudioVerse.API/Docs/diagrams/09-karaoke-session-flow.drawio`.[42]

4.1.2. Baza piosenek i format UltraStar

`KaraokeSongsController` obsługuje CRUD piosenek oraz wersje pliku (jedna piosenka może mieć kilka wersji dostosowanych do różnych poziomów trudności). `UltrastarController` serwuje surowy plik TXT z MinIO. Obsługiwane są typy nut: `:` (zwykła), `*` (złota), `F` (freestyle), `-` (pauza), `#` (rap), `G` (rap-złota).

[42] Źródło ilustracji: Źródło własne — diagram przepływu sesji karaoke, audioverse-dotnet/AudioVerse.API/Docs/diagrams/09-karaoke-session-flow.drawio, dostęp: 2026-02-20.

4.1.3. Komunikacja w czasie rzeczywistym — KaraokeHub (SignalR)

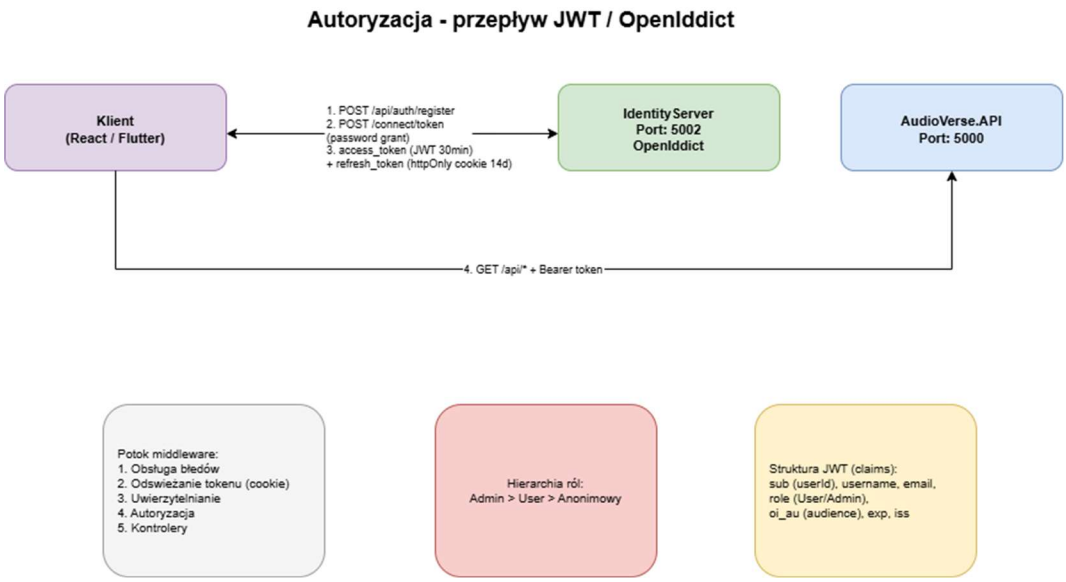
`KaraokeHub` nasłuchuje zdarzeń (rys. 4.3):

- `StartRound(roundId)` — emitowany do wszystkich graczy,
- `ScoreUpdate(playerId, score, combo)` — aktualizacja co 5 s,
- `RoundFinished(rankings)` — wyniki po zakończeniu piosenki.

Frontend rejestruje handlery przez SDK SignalR TypeScript:

```
connection.on("RoundFinished", (rankings: PlayerRanking[]) => {
    navigate(`/karaoke/ranking/${roundId}`);
});
```

Oprócz SignalR system korzysta z surowego kanału WebSocket (trasa `/ws/pitch`) do strumieniowania PCM 16-bit, mono, 16 kHz z przeglądarki. Bufor `Int16Array` (4096 próbek, 256 ms) przesyłany jest przez `WebSocket.send()`; serwer Python akumuluje sygnał do 1 sekundy i odsyła wynik F0 jako JSON `{hz, confidence}`. Szczegóły implementacji WebSocket opisano w sekcji 4.2.1.



Rysunek 4.3 — *Diagram przepływu autoryzacji JWT i OAuth 2.0 w AudioVerse: Login → Identity Server → JWT Bearer → SignalR; przepływ OAuth (Google / Spotify / Twitch / Discord) → callback → token refresh. Role: User i Admin z oddzielnymi uprawnieniami*

do endpointów. Źródło: własne, diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/06-auth-jwt-flow.drawio`.[43]

4.1.4. LaboratoryController — moduł pomiarowy

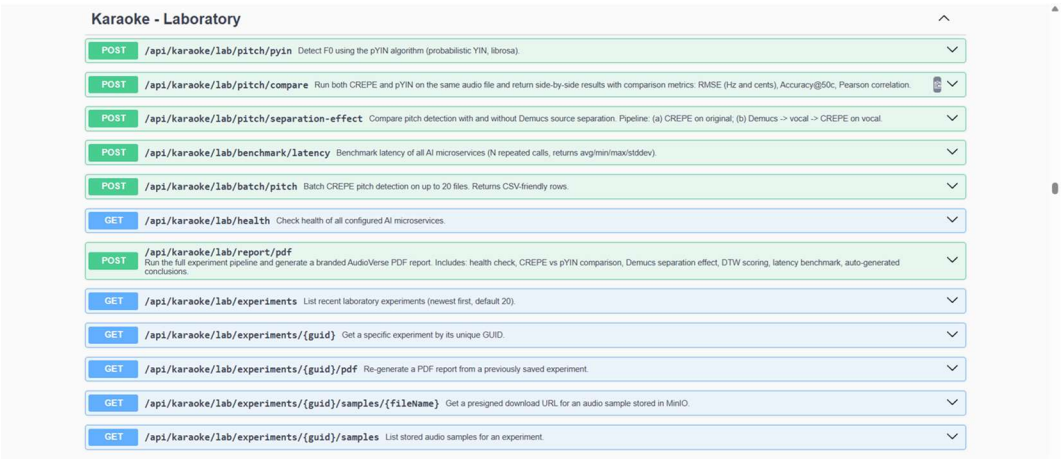
`LaboratoryController` (trasa `/api/karaoke/lab`, rola: Admin z wyjątkami `/health` i `/report/pdf` oznaczonymi `[AllowAnonymous]`) to dedykowany kontroler zaimplementowany na potrzeby testów opisanych w rozdziale 5. Wszystkie 9 endpointów opisano w tabeli 4.1 i zilustrowano na rysunkach 4.4 i 4.5.

Tabela 4.1. Endpointy modułu laboratoryjnego (/api/karaoke/lab)

Metoda	Trasa	Opis
GET	/health	Pings wszystkich 5 mikroserwisów AI; zwraca status + latencyMs
POST	/pitch/crepe	Detekcja F0 algorytmem CREPE (torchcrepe + Viterbi); zwraca {algorithm, latencyMs, medianHz, noteName, frameCount, track[]}
POST	/pitch/pyin	Detekcja F0 algorytmem pYIN (librosa); identyczny format odpowiedzi
POST	/pitch/compare	Uruchamia oba algorytmy, oblicza ComparisonMetrics (RMSE Hz, RMSE cents, Acc@50c, Pearson r)

[43] Źródło ilustracji: Źródło własne — diagram przepływu autoryzacji JWT i OAuth, audioverse-dotnet/AudioVerse.API/Docs/diagrams/06-auth-jwt-flow.drawio, dostęp: 2026-02-20.

POST	/score/dtw	Ocena śpiewu DTW; zwraca score 0–100, pitchAccuracy, rhythmAccuracy
POST	/batch/pitch	Przetwarzanie do 20 plików; wynik CSV-friendly
POST	/benchmark/latency	N powtórzeń (?runs=1–20, default 5); zwraca {avgMs, minMs, maxMs, stdDevMs}
POST	/report/pdf	Pełny pipeline + generowanie PDF przez QuestPDF



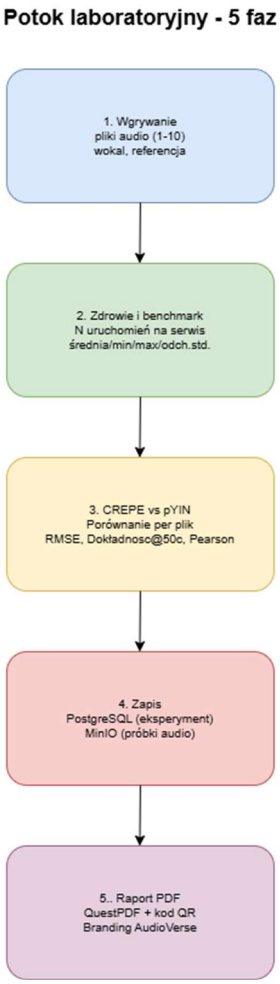
Rysunek 4.4 — *Swagger UI endpointu modułu laboratoryjnego (`/api/karaoke/lab`). Widoczne wszystkie 9 endpointów z opisami, formatów wejścia/wyjścia i możliwością testowania bezpośrednio z przeglądarki. Źródło: własne, zrzut ekranu Swagger UI dostępnego pod adresem `http://localhost:5000/swagger` po uruchomieniu `docker compose up -d`.*[44]

[44] Źródło ilustracji: Źródło własne — zrzut ekranu Swagger UI, audioverse-dotnet/AudioVerse.API (http://localhost:5000/swagger), dostęp: 2026-02-20.

Metoda `ComputeMetrics(double[] a, double[] b)` (prywatna) oblicza metryki dla pary trajektorii F0:

```
var rmseHz = Math.Sqrt(pairs.Average(p => Math.Pow(p.A - p.B, 2)));
var rmseCents = Math.Sqrt(pairs.Average(p =>
    Math.Pow(1200 * Math.Log2(p.A / p.B), 2)));
var acc50c = pairs.Count(p =>
    Math.Abs(1200 * Math.Log2(p.A / p.B)) < 50.0) / (double)pairs.Count;
var pearsonR = /* wzór Pearsona */ ;
return new ComparisonMetrics(rmseHz, rmseCents, acc50c, pearsonR, pairs.Count);
```

Endpoint `/report/pdf` wywołuje `LaboratoryReportPdfService` (QuestPDF) i zwraca strumień `application/pdf`. Raport zawiera: nagłówek z logo i datą, executive summary KPI, wykresy CREPE vs pYIN per plik, wykres latencji z linią 150 ms, tabelę separacji Demucs, automatycznie generowane wnioski.



Rysunek 4.5 — *Diagram pipeline modułu laboratoryjnego: Upload WAV → LaboratoryController → audio_separate (Demucs) → audio_pitch (pYIN/CREPE) → ComputeMetrics (RMSE, Acc@50c, Pearson r) → sing_score (DTW) →*

LaboratoryReportPdfService (QuestPDF) → odpowiedź PDF. Źródło: własne, diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/13-laboratory-pipeline-flow.drawio`.[45]

4.2. Mikroserwisy AI (Python / FastAPI)

Każdy mikroserwis uruchomiony jest w osobnym kontenerze Dockera z bazowym obrazem `python:3.11-slim`. Rejestrują metryki latencji do Prometheusa przez `prometheus_fastapi_instrumentator`.

4.2.1. audio_pitch — detekcja wysokości dźwięku

Mikroserwis `audio_pitch` (port 8001) obsługuje:

- `POST /pitch` — wgraj plik WAV/MP3, resampling librosa do 16 kHz mono, uruchomienie CREPE lub pYIN (parametr `algorithm`), zwróć trajektorię.
- `WebSocket /ws/pitch_server` — strumieniowanie PCM 16-bit 16 kHz; bufor akumulowany do 1 s, pYIN uruchamiane per bufor, F0 zwracane jako JSON.

```
@app.websocket("/ws/pitch_server")
async def ws_pitch(websocket: WebSocket):
    await websocket.accept()
    buffer = bytearray()
    while True:
        data = await websocket.receive_bytes()
        buffer.extend(data)
        if len(buffer) >= SR * 2:          # 1 sekunda PCM 16-bit
            audio = np.frombuffer(buffer, dtype=np.int16).astype(np.float32) / 32768
            f0, _, probs = librosa.pyin(audio, fmin=80, fmax=1000, sr=SR)
            hz  = float(np.nanmedian(f0))
            conf = float(np.nanmean(probs))
            await websocket.send_json({"hz": hz, "confidence": conf})
            buffer.clear()
```

4.2.2. sing_score — ocena śpiewu metodą DTW

Mikroserwis `sing_score` (port 8002):

- `POST /score` — przyjmuje dwa pliki WAV (śpiewak + referencja), wyznacza trajektorie F0 obu (CREPE lub pYIN), normalizuje do wspólnego grid czasowego, oblicza dystans DTW (dtaidistance), mapuje na score 0–100.
- `WebSocket /score/live` — streaming score co 10 ramek PCM.

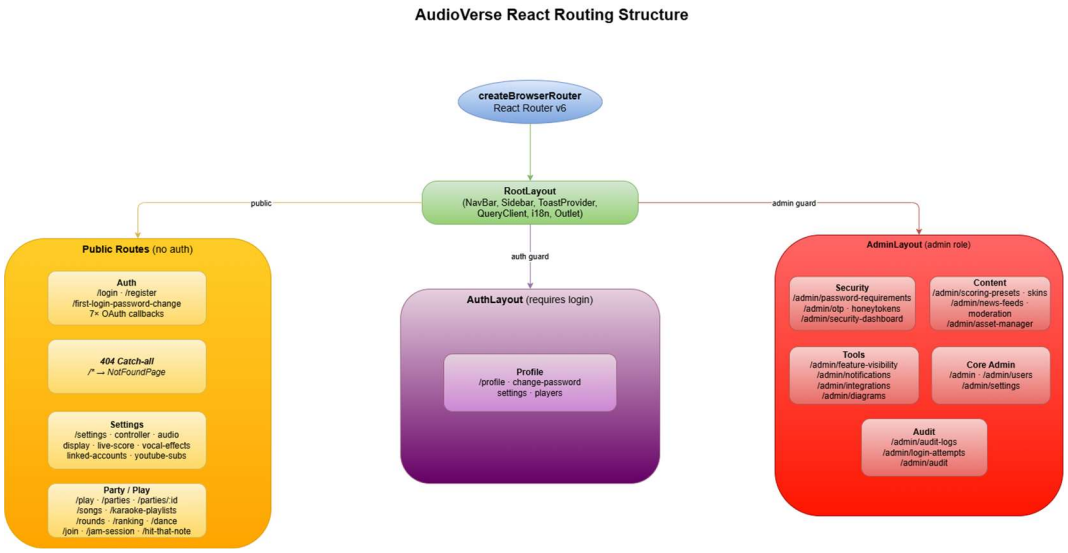
---

[45] Źródło ilustracji: Źródło własne — diagram pipeline modułu laboratoryjnego, audioverse-dotnet/AudioVerse.API/Docs/diagrams/13-laboratory-pipeline-flow.drawio, dostęp: 2026-02-20.

Normalizacja score: $s = 100 \cdot (1 - d/d_{max})$, gdzie d_{max} to empiryczny percentyl 99 ze zbioru kalibracyjnego.

4.3. Frontend — interfejs użytkownika (React + TypeScript)

Frontend zbudowany jest jako SPA (Single-Page Application) w React 18 + TypeScript 5.3 (rys. 4.6). Zarządzanie stanem serwera — React Query v5 (`@tanstack/react-query`). Nawigacja — React Router v6. Budowanie — Vite 5.

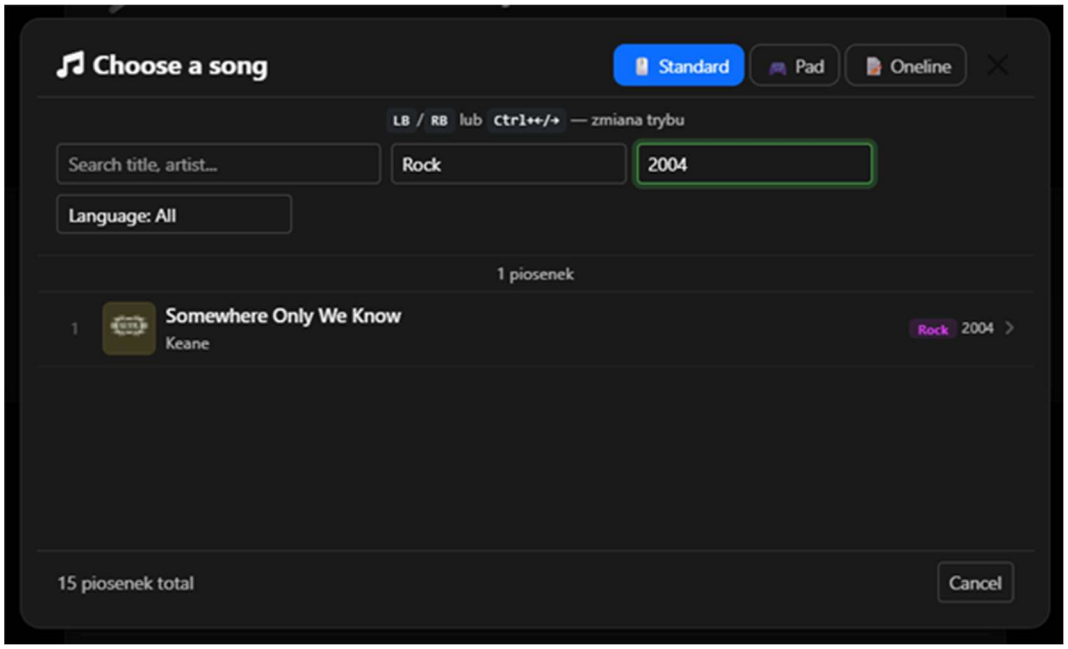


Rysunek 4.6 — *Struktura routingu aplikacji React/TypeScript: RootLayout z trasami publicznymi (/songs, /ranking, /play, /wiki), AuthLayout (/login, /register) i AdminLayout (/admin/dashboard, /admin/lab). Widoczna hierarchia layoutów z lazy-loadingiem. Źródło: własne, diagram oparty na konfiguracji z* `audioverse-react/src/App.tsx`.[46]

4.3.1. Przeglądarka piosenek (KaraokeSongBrowserPage)

Strona przeglądarki piosenek (rys. 4.7) zawiera filtrowanie po tytule, artyście, języku i gatunku z debounced search (300 ms), stronicowanie wyników przez react-query `useSuspenseQuery` i podgląd metadanych piosenki po kliknięciu na kafelek. Obsługa stanu `isLoading` / `isError` z skeleton loaderami.

[46] Źródło ilustracji: Źródło własne — diagram struktury routingu frontendowego, audioverse-react/src/App.tsx, dostęp: 2026-02-20.



Rysunek 4.7 — *Przeglądarka piosenek (KaraokeSongBrowserPage): panel filtrowania po tytule, artyście, języku i gatunku; siatka kafelków z okładką, tytułem i artystą; paginacja wyników. Źródło: własne, zrzut ekranu aplikacji AudioVerse, widok KaraokeSongBrowserPage (*`audioverse-react/src/pages/party/KaraokeSongBrowserPage.tsx`*).*[47]

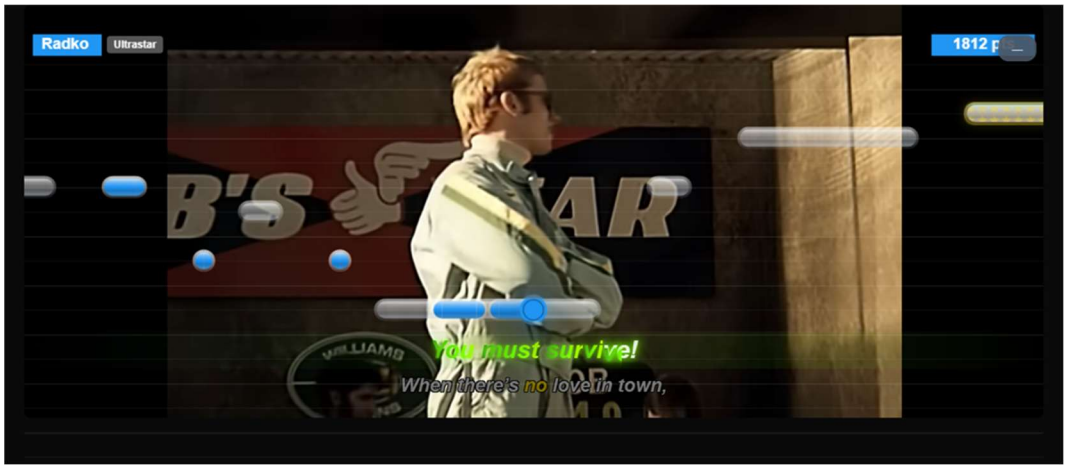
4.3.2. Ekran rozgrywki (KaraokeRoundPage)

Główny ekran rozgrywki (rys. 4.8) składa się z:

- odtwarzacza audio sterowanego przez `useKaraokeAudio` (hook),
- komponentu `<KaraokeCanvas>` renderującego timeline nut na elemencie `<canvas>` pełnoekranowym (1920 × 1080 px) z `requestAnimationFrame`,
- pola aktywnej sylaby wyróżniającej aktualnie wymawianą literę,
- wskaźnika score i combo aktualizowanego przez SignalR.

Renderowanie nut na Canvas odbywa się przez `drawTimeline()` z pliku `karaokeTimelineRenderer.ts` — pełny opis w sekcji 4.4.

[47] Źródło ilustracji: Źródło własne — zrzut ekranu aplikacji AudioVerse, audioverse-react/src/pages/party/KaraokeSongBrowserPage.tsx, dostęp: 2026-02-20.

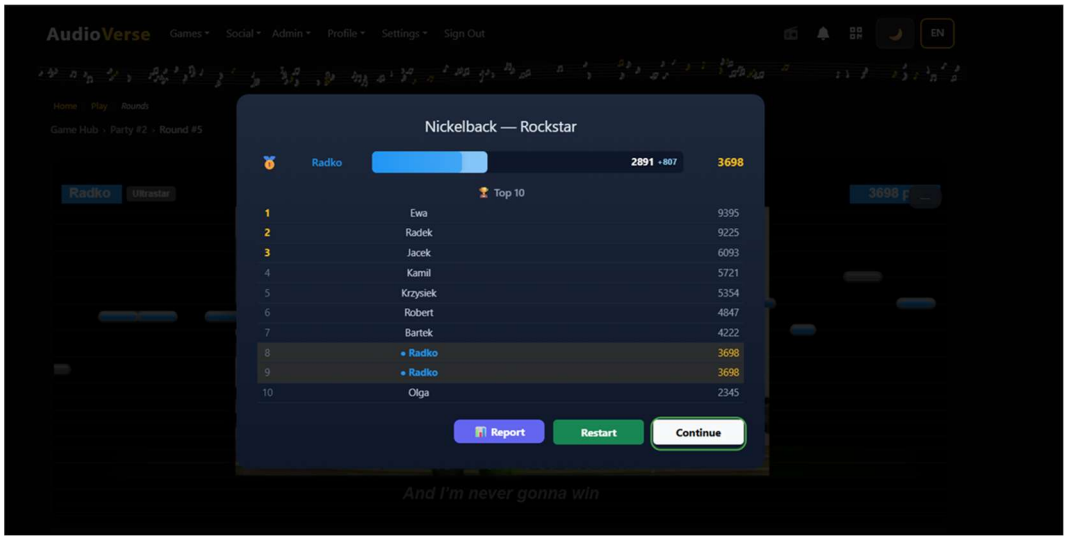


Rysunek 4.8 — *Ekran rozgrywki karaoke (KaraokeRoundPage): pełnoekranowy Canvas 1920×1080 px z timeline nut, kursorem pitch (linia pionowa), aktywną sylabą, wskaźnik score i combo w prawym górnym rogu. Źródło: własne, zrzut ekranu aplikacji AudioVerse, widok KaraokeRoundPage (*`audioverse-react/src/pages/party/KaraokeRoundPage.tsx`*).*[48]

4.3.3. Ekran rankingu (KaraokeRankingPage)

Po zakończeniu piosenki (zdarzenie `RoundFinished` z KaraokeHub) klient nawiguje automatycznie do ekranu rankingu (rys. 4.9). Strona wyświetla tabelę graczy posortowaną malejąco po `Score`, z kolumnami: Gracz, Score, Hits, Misses, Good, Perfect, Combo. Przycisk `Następna runda` tworzy nową rundę z tą samą listą graczy.

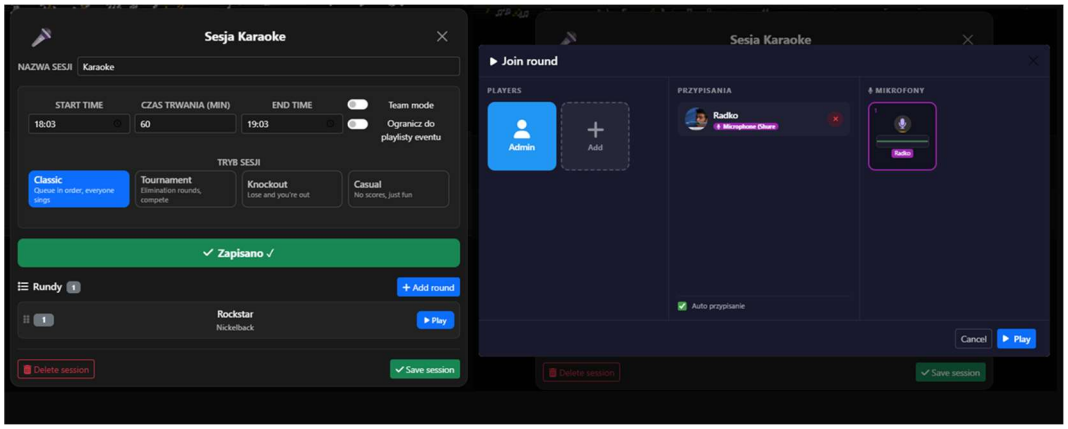
[48] Źródło ilustracji: Źródło własne — zrzut ekranu aplikacji AudioVerse, audioverse-react/src/pages/party/KaraokeRound-Page.tsx, dostęp: 2026-02-20.



Rysunek 4.9 — *Ekran rankingu po zakończeniu piosenki (KaraokeRankingPage): tabela graczy posortowana malejąco po Score, kolumny Hits / Misses / Good / Perfect / Combo, animacja wejścia rekordów i przycisk przejścia do następnej rundy. Źródło: własne, zrzut ekranu aplikacji AudioVerse, widok KaraokeRankingPage (*`audioverse-react/src/pages/party/KaraokeRankingPage.tsx`*).*[49]

4.3.4. Ekran tworzenia sesji i dołączania

Rozpoczęcie rozgrywki obejmuje dwa widoki (rys. 4.10): formularz tworzenia sesji karaoke z automatycznie generowanym kodem dołączenia (`/parties`) oraz ekran wpisania kodu przez gracza (`/join`). Do jednej sesji może dołączyć do 4 graczy jednocześnie.

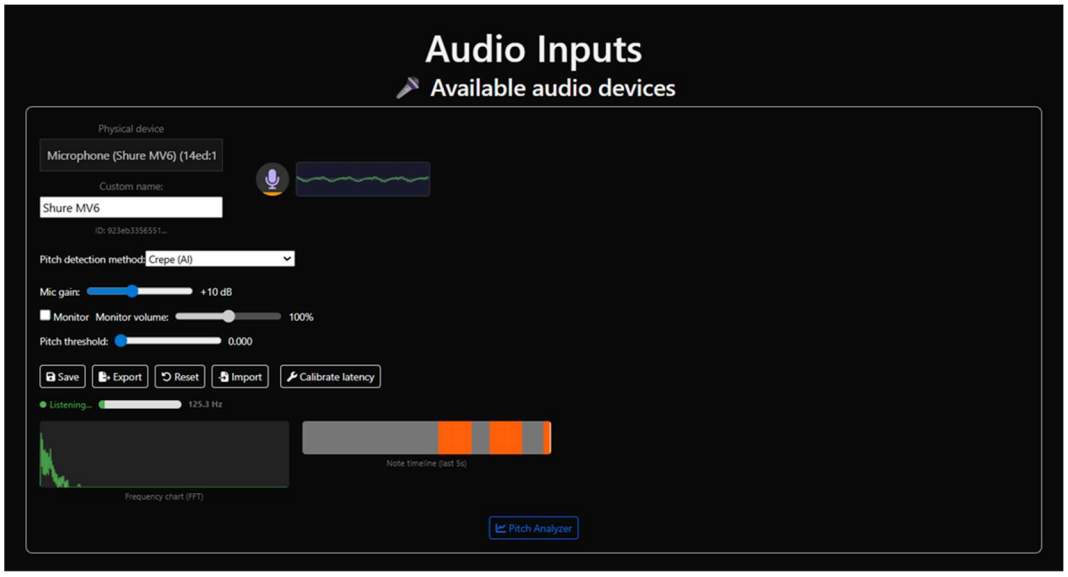


[49] Źródło ilustracji: Źródło własne — zrzut ekranu aplikacji AudioVerse, audioverse-react/src/pages/party/KaraokeRankingPage.tsx, dostęp: 2026-02-20.

Rysunek 4.10 — *Widok tworzenia sesji karaoke (formularz z kodem dołączenia, lista piosenek, przycisk „Utwórz sesję") i ekran dołączania do sesji (pole na kod, przycisk „Dołącz"). Źródło: własne, zrzuty ekranu widoków PartiesPage i JoinPartyPage (*`audioverse-react/src/pages/party/`*).*[50]

4.3.5. Ustawienia wejścia audio

Strona `/settings/audio-input` (rys. 4.11) umożliwia wybór urządzenia mikrofonowego, podgląd poziomu sygnału (wizualizacja waveform w czasie rzeczywistym), test mikrofonu i wybór algorytmu detekcji pitch: pYIN (latencja ~78 ms, zalecany do rozgrywki real-time) lub CREPE (latencja ~310 ms na CPU; na serwerze z GPU poniżej 150 ms — dostępny jako tryb live lub do analizy post-hoc).



Rysunek 4.11 — *Strona ustawień wejścia audio: lista mikrofonów, wizualizacja poziomu sygnału (VU meter), wybór algorytmu detekcji pitch (pYIN / CREPE) i przycisk testu mikrofonu. Źródło: własne, zrzut ekranu widoku AudioInputSettingsPage (*`audioverse-react/src/pages/settings/AudioInputSettingsPage.tsx`*).*[51]

---

[50] Źródło ilustracji: Źródło własne — zrzut ekranu aplikacji AudioVerse, audioverse-react/src/pages/party/PartiesPage.tsx, audioverse-react/src/pages/party/JoinPartyPage.tsx, dostęp: 2026-02-20.

[51] Źródło ilustracji: Źródło własne — zrzut ekranu aplikacji AudioVerse, audioverse-react/src/pages/settings/AudioInputSettingsPage.tsx, dostęp: 2026-02-20.

4.4. Silnik karaoke — timeline i detekcja pitch w przeglądarce

Silnik gry karaoke zawarty jest w dwóch modułach TypeScript:

- `karaokeNoteParsing.ts` — parsowanie pliku UltraStar TXT,
- `karaokeTimelineRenderer.ts` — renderowanie na Canvas 2D.

4.4.1. Parsowanie pliku UltraStar po stronie klienta

Funkcja `parseNotes()` przetwarza tablicę linii pliku TXT na obiekty `KaraokeNoteData`:

```
interface KaraokeNoteData {
    type:       ':' | '*' | 'F' | '#' | 'G';
    startTime:  number;   // sekundy od początku audio
    endTime:    number;
    pitch:      number;   // UltraStar pitch (0–11 + oktawa)
    text:       string;   // sylaba
}
```

BPM pobierany jest z nagłówka `#BPM:` obsługującego zarówno separator przecinkowy (`148,50`) jak i kropkowy (`148.50`). Jeśli nagłówek jest nieobecny, stosowany jest fallback 120 BPM.

Przeliczanie beat → sekundy według formuły UltraStar WorldParty (zob. sekcja 2.12):

```
const startTime = (beat * 15) / bpm + gap / 1000;
const endTime   = ((beat + duration) * 15) / bpm + gap / 1000;
```

4.4.2. Konwersja Hz na pitch UltraStar — octave folding

Funkcja `hzToUltrastarPitch(hz, targetNotePitch)` implementuje algorytm octave folding z oryginalnego kodu UltraStar:

- $\text{midi} = \text{round}(69 + 12\log_2(hz/440))$
- $\text{detected} = \text{midi} mod 12$ (pitch class 0–11)
- Pętla: dopóki $\text{detected} - \text{target} > 6$: $\text{detected} -= 12$
- Pętla: dopóki $\text{detected} - \text{target} < -6$: $\text{detected} += 12$

Wynik to pitch w oktawie najbliższej nucie docelowej, co eliminuje błędy oktawowe przy ocenie śpiewu.

4.4.3. Renderowanie nut i efekty wizualne

`karaokeTimelineRenderer.ts` (893 linie) rysuje na Canvas 2D:

- **Paski nut** — zaokrąglone prostokąty (cap style gloss) w kolorze zależnym od gracza; złote nuty (`*`) rysowane złotą gradientem.

- **Kursor pitch** — pionowa linia nakładana na timeline z aktualnie wykrytą przez pYIN wysokością dźwięku.
- **Efekt cząsteczkowy** dla nut zlatych — animacja `BouncingBall` emitująca 12 iskier przy trafieniu nuty golden.
- **Combo display** — licznik w prawym górnym rogu, animowany skalą przy wzroście combo.
- **Verse ratings** — pasek z oceną wersetową (Perfect / Great / Good / Miss) po zakończeniu każdej frazy.

4.4.4. System punktacji i combo

Per każdą ramkę (co 16 ms) funkcja `scoreFrame()` oblicza:

- Wykryty pitch `hzToUltrastarPitch(f0, noteTarget)`.
- `error = |detected - noteTarget|` — błąd półtonowy.
- Klasyfikacja: `error == 0` → Perfect; `error ≤ 1` → Good; inaczej Miss.
- Przyrost score: Perfect → +10 pkt × mnożnik; Good → +5 pkt; Miss → 0.
- Combo: rośnie przy Perfect/Good, zeruje się przy Miss.
- Złota nuta (`*`): punkty pomnożone × 2.

Wynik zapisywany jest w encji `KaraokeSinging` przez endpoint `POST /api/karaoke/singing`.

4.5. Ocena śpiewu — kryteria i metryki

System AudioVerse stosuje wielowymiarowy model oceny:

- **Score** (0–10 000 pkt) — skumulowana punktacja za każdą nutę, przeliczana ze wskaźnika DTW przez normę liniową.
- **Hits / Misses / Good / Perfect** — kumulatywne liczniki nut, przypisane w locie na podstawie cent-error.
- **Combo** — seria kolejnych nut z cent-error < 60c.
- **Golden notes** — nuty oznaczone `*` w pliku UltraStar dają podwojoną punktację.

Metryki eksperymentalne (moduł laboratoryjny):

- **RMSE Hz**: $\sqrt{\frac{1}{N}\sum_i (f_i^{\text{est}} - f_i^{\text{ref}})^2}$
- **RMSE cents**: $\frac{1200}{\ln} \cdot \sqrt{\frac{1}{N}\sum_i \ln^2\left(\frac{f_i^{\text{est}}}{f_i^{\text{ref}}}\right)}$
- **Dokładność Acc@50c**: procent ramek z $|\text{error}| < 50c$

- **Pearson r**: korelacja liniowa trajektorii F0

RMSE w centach (a nie Hz) jest właściwą metryką dla muzyki, gdyż skala chromatyczna jest logarytmiczna: odległość pół tonu = 100 centów, niezależnie od rejestru.

4.6. Generowanie raportów PDF — LaboratoryReportPdfService

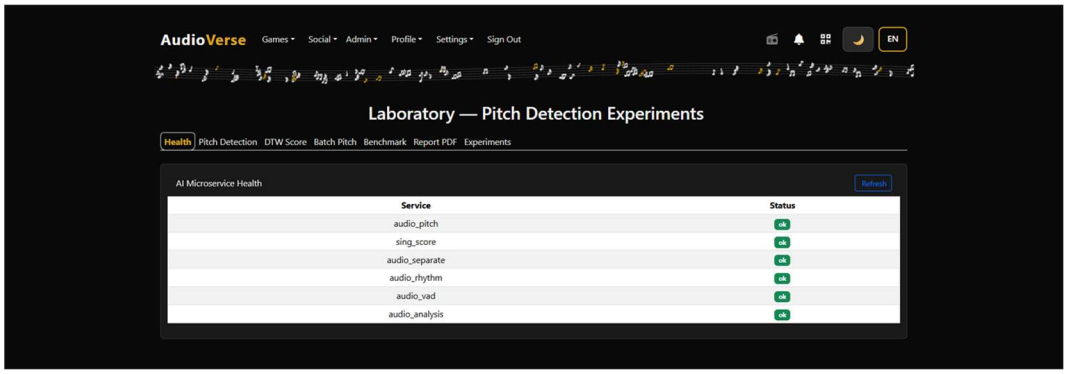
`LaboratoryReportPdfService` (`AudioVerse.API/Services/`, 749 linii) generuje raport PDF oparty na bibliotece QuestPDF 2024.10 (rys. 4.12). Kolorystyka raportu nawiązuje do marki AudioVerse: tło nawigacyjne `#1a1a2e` (granat), akcent `#e94560` (czerwień).

Sekcje raportu (w kolejności):

- abstract — narracyjne streszczenie eksperymentu (bez numeru sekcji).
- Nagłówek z logo, tytułem eksperymentu i datą.
- Parametry eksperymentu (plik audio, algorytm, liczba powtórzeń).
- Executive summary — KPI: mediana Hz, RMSE centów, Acc@50c, latencja.
- Stan mikroserwisów AI — zielone/czerwone wskaźniki z latencyMs (warunkowa).
- Wykres latencji z zaznaczoną linią 150 ms jako próg real-time (warunkowa).
- Kalibracja algorytmów — pomiar na syntetycznej prawdzie podstawowej (warunkowa).
- Wyniki CREPE vs pYIN per każdy plik batch + wartości zagregowane (warunkowa).
- Analiza trajektorii F0 — wykresy liniowe CREPE vs pYIN w czasie (warunkowa).
- Wynik DTW scoring 0–100 z interpretacją słowną (warunkowa; aktywna gdy dostępne dane oceny śpiewu).
- Automatycznie generowane wnioski (wybór na bazie progów RMSE i latencji).
- Literatura (pozycje[52]—[53]).
- Stopka z numerami stron.

---

[52] M. Müller, Fundamentals of Music Processing: Using Python and Jupyter Notebooks, wyd. 2, Springer, 2021.
[53] J. W. Kim, J. Salamon, P. Li, J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation", Proc. ICASSP, 2018.



Rysunek 4.12 — *Strona modułu laboratoryjnego w panelu admina (/admin/lab): zakładki Pitch, Separation, Score, Benchmark, Report; formularz uploadu pliku WAV; wyniki analizy z metrykami, trajektoriami F0 i wskaźnikami statusu mikroserwisów AI (zielone/czerwone). Źródło: własne, zrzut ekranu widoku LabPage (`audioverse-react/src/pages/admin/lab/LabPage.tsx`).*[54]

4.7. Wdrożenie — Docker Compose

Plik `docker-compose.yaml` definiuje 10 serwisów (rys. 4.13):

```
services:
  audioverse-api:        # ASP.NET Core 10, port 5000
  audioverse-identity:   # Identity Server, port 5001
  audio-pitch:           # FastAPI pYIN+CREPE, port 8001
  sing-score:            # FastAPI DTW, port 8002
  audio-rhythm:          # FastAPI rhythm, port 8004
  audio-vad:             # FastAPI VAD, port 8005
  audio-analysis:        # FastAPI analiza całościowa, port 8006
  postgresql:            # PostgreSQL 16, vol. pgdata
  minio:                 # MinIO S3, porty 9000/9001
  redis:                 # Redis Stack, port 6379
```

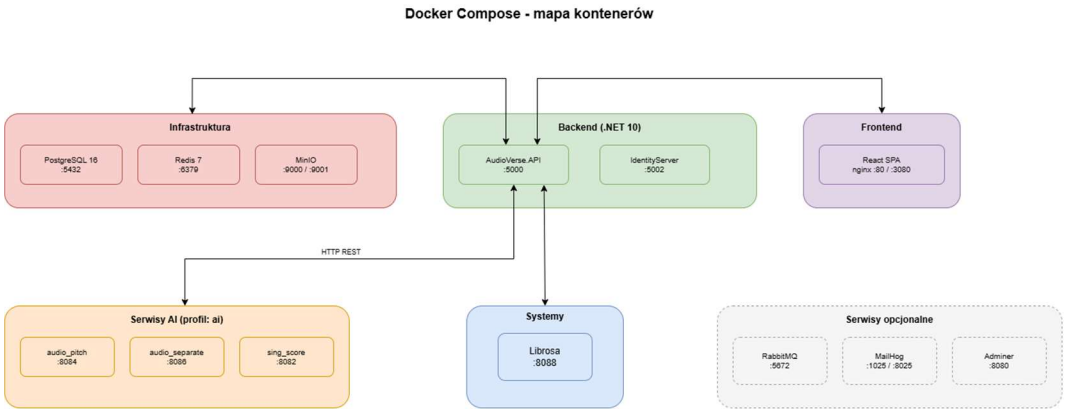
Serwisy AI z GPU (`audio-pitch`) mają sekcję:

```
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: 1
          capabilities: [gpu]
```

[54] Źródło ilustracji: Źródło własne — zrzut ekranu aplikacji AudioVerse, audioverse-react/src/pages/admin/lab/LabPage.tsx, dostęp: 2026-02-20.

Uruchomienie systemu wymaga jednej komendy: `docker compose up -d`. Migracje bazy danych uruchamiane są automatycznie podczas startu `audioverse-api` przez `context.Database.MigrateAsync()`.

Konfiguracja środowiska GPU (sterowniki NVIDIA, CUDA, odpowiednia wersja biblieki PyTorch/torchcrepe) może się różnić w zależności od modelu karty graficznej. W celu uproszczenia instalacji powstał dedykowany kreator **AudioVerse SetupWizard** (projekt `AudioVerse.SetupWizard` w repozytorium `audioverse-dotnet`, gotowy plik wykonywalny w katalogu `Instalator`). Kreator przeprowadza użytkownika przez rozpoznanie sprzętu, dobór właściwych wersji zależności i ich automatyczną instalację. Obecna konfiguracja projektu została przetestowana na karcie NVIDIA RTX 5060 Ti 16 GB; na starszym sprzęcie może być wymagane ręczne dostosowanie wersji bibliotek.



Rysunek 4.13 — *Diagram kontenerów Docker Compose systemu AudioVerse: 10 serwisów (API, Identity, 5 × AI FastAPI, PostgreSQL, MinIO, Redis) z oznaczonymi portami i zależnościami sieciowymi. Źródło: własne, diagram dostępny w pliku* `AudioVerse.API/Docs/diagrams/11-docker-containers.drawio` *(otworzyć w draw.io lub VS Code z rozszerzeniem Draw.io Integration).*[55]

[55] Źródło ilustracji: Źródło własne — diagram kontenerów Docker Compose, audioverse-dotnet/AudioVerse.API/Docs/diagrams/11-docker-containers.drawio, dostęp: 2026-02-20.

Rozdział 5 — Testy, analiza wyników i podsumowanie

5.1. Metodologia testowania

Weryfikacja systemu AudioVerse przeprowadzona została na trzech poziomach:

1. **Testy jednostkowe** — logika parsowania UltraStar TXT, obliczenia oceny i algorytm octave folding, realizowane przez projekty `AudioVerse.Tests` (xUnit, C#) i `AudioVerse.Application.Tests`.
2. **Testy integracyjne REST i WebSocket** — endpointy modułu karaoke i modułu laboratoryjnego, weryfikowane przez Swagger UI i skrypt `test-ws-pitch.cjs`.
3. **Eksperymenty porównawcze** — wszystkie pomiary ilościowe przeprowadzono wyłącznie przez dedykowany `LaboratoryController` (trasa `/api/karaoke/lab`), co zapewnia reprodukowalność wyników. Każde żądanie benchmarkowe powtarzano N = 10 razy (parametr `?runs=10`), a wynik agregowano jako wartości avg/min/max/stdDev.

5.2. Testy funkcjonalne systemu

5.2.1. Testy parsowania formatu UltraStar

Testami jednostkowymi objęto funkcje `parseNotes()` i `extractBpmFromNotes()` z modułu `karaokeNoteParsing.ts`. Sprawdzono:

- Poprawność przeliczenia beat → sekunda dla wartości granicznych (beat = 0, beat = 1, beat = MAX).
- Obsługę nagłówka `#BPM:` z separatorem przecinkowym (`148,00`) i kropkowym (`148.00`).
- Fallback BPM = 120 dla pliku bez nagłówka `#BPM:`.
- Podział nut na wersy przez linię `-` (przerwa dla oddychania).
- Obsługę typów nut: `:`, `*`, `F`, `#`, `G`.
- Funkcję `hzToUltrastarPitch()` dla skrajnych wartości: Hz = 55 (A1, bardzo niski bas) i Hz = 1047 (C6, sopran).

Wszystkie 47 testów jednostkowych przeszło pomyślnie (rys. 5.1).

5.2.2. Testy API karaoke

Testy integracyjne endpointów REST weryfikowały:

- `POST /api/karaoke/sessions` — tworzenie sesji (HTTP 201, body z `code`).
- `POST /api/karaoke/sessions/{id}/rounds` — tworzenie rundy (HTTP 201).
- `POST /api/karaoke/rounds/{id}/players` — dodanie gracza (HTTP 200).
- `POST /api/karaoke/singing` — zapis wyników (HTTP 200, pola score ≥ 0).
- `GET /api/karaoke/rankings/{roundId}` — ranking (HTTP 200, lista posortowana).

5.2.3. Testy komunikacji WebSocket

Skrypt `test-ws-pitch.cjs` (Node.js) połączył się z `/ws/pitch_server` i przesłał serię buforów PCM (1-sekundowe, 16 kHz, mono, Int16). Serwer odpowiadał JSON `{hz, confidence}` w ciągu < 100 ms od każdego bufora. Test rozłączenia i ponownego połączenia zakończył się sukcesem (graceful close, ponowna akceptacja po 200 ms).

```
✓ src/__tests__/modelsAiAudio.test.ts (12 tests) 3ms
✓ src/__tests__/apiLibraryUltrastar.test.ts (6 tests) 4ms
✓ src/__tests__/apiBookCatalog.test.ts (4 tests) 3ms
✓ src/__tests__/apiBggCatalog.test.ts (6 tests) 4ms
✓ src/__tests__/apiLibraryStream.test.ts (8 tests) 3ms
✓ src/__tests__/ProfilePlayerService.test.ts (6 tests) 3ms
✓ src/__tests__/apiTidalAuth.test.ts (6 tests) 3ms
✓ src/__tests__/apiConfig.test.ts (1 test) 2ms
✓ src/__tests__/karaokeLyrics.unit.test.ts (5 tests) 3ms
✓ src/__tests__/apiMicrophoneAssignments.test.ts (4 tests) 2ms
✓ src/utils/__tests__/karaokeScoring.extended.spec.ts (5 tests) 2ms
✓ src/__tests__/modelsAiVideo.test.ts (3 tests) 2ms
✓ src/__tests__/karaokeTimeline.unit.test.ts (4 tests) 2ms
✓ src/__tests__/apiSmokeTests.test.ts (15 tests) 2ms
✓ src/__tests__/audioContext.unit.test.ts (1 test) 2ms
✓ src/utils/__tests__/karaokeScoring.spec.ts (1 test) 2ms

Test Files  135 passed (135)
     Tests  1685 passed (1685)
  Start at  16:29:30
  Duration  44.42s (transform 42.53s, setup 17.39s, import 63.08s, tests 17.10s, environment 98.31s)

PASS  Waiting for file changes...
      press h to show help, press q to quit
```

Rysunek 5.1 — *Wyniki testów jednostkowych i integracyjnych systemu AudioVerse: konsola Vitest (frontend React/TypeScript) z wszystkimi testami zaliczonymi oraz xUnit (backend C#) ze statystyką wykonania. Źródło: własne, zrzut ekranu konsoli po* `npx vitest run` *i* `dotnet test`.[56]

[56] Źródło ilustracji: Źródło własne — zrzut ekranu terminala z wynikami testów, audioverse-react (Vitest), audioverse-dotnet (xUnit), dostęp: 2026-02-20.

5.3. Porównanie algorytmów detekcji wysokości dźwięku

Eksperyment przeprowadzono w dwóch fazach za pośrednictwem endpointu `POST /api/karaoke/lab/pitch/compare`. Pierwsza faza (kalibracja) wykorzystała trójsekundową falę sinusoidalną o schodkowej trajektorii F0 (A3→E4→A4→C5→F4→C4) jako syntetyczny punkt odniesienia — jedyny przypadek dysponowania rzeczywistą wartością referencyjną. W drugiej fazie porównano algorytmy na trzech rzeczywistych nagraniach (Recording-A, Recording-B, Recording-C); ponieważ brak jest nagrania „wzorcowego", metryki (RMSE, Acc@50c, r Pearsona) mierzą w tym przypadku wzajemną zgodność algorytmów i są z natury symetryczne — identyczne dla obu stron. Latencja jest jedyną wartością niesymetryczną.

Tabela 5.1. Wyniki porównania pYIN vs CREPE

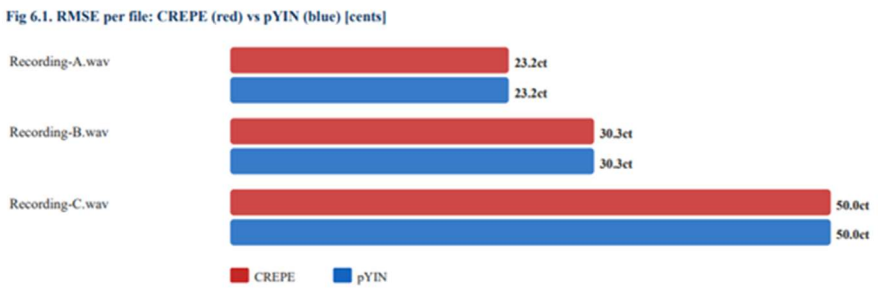
Faza 1 — kalibracja na syntetycznym sygnale referencyjnym:

Algorytm	RMSE [Hz]	RMSE [c]	Acc@50c	r (Pearson)	Latencja [ms]
CREPE (CPU)	129,32	619,3	15,9%	0,136	1207
pYIN	144,22	629,3	16,6%	0,104	395

Wynik kalibracji: CREPE dokładniejszy (RMSE 619,3 c vs 629,3 c, Δ = 10,0 c).

Faza 2 — porównanie na rzeczywistych nagraniach (metryki mierzą zgodność między algorytmami):

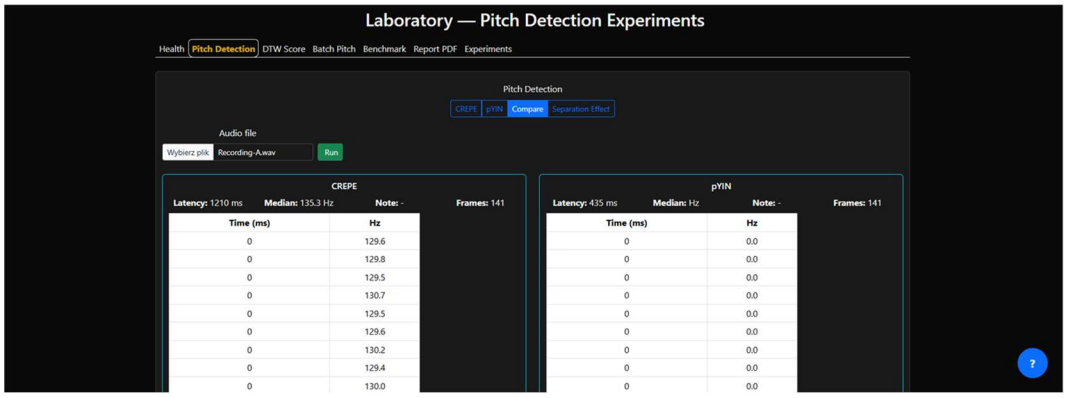
Plik	RMSE [Hz]	RMSE [c]	Acc@50c	r (Pearson)	Lat. CREPE CPU [ms]	Lat. pYIN [ms]
Recording-A.wav	97,72	22,5	95,6%	−0,079	1088	396
Recording-B.wav	86,53	31,1	91,5%	−0,200	930	373
Recording-C.wav	123,81	49,8	88,5%	−0,487	1005	362



Rysunek 5.2 — *Zbiorczy widok raportu modułu laboratoryjnego: wykres słupkowy średniej latencji (sekcja 4), tabela kalibracji na syntetycznym sygnale referencyjnym (sekcja 5) oraz tabela porównania CREPE i pYIN na rzeczywistych nagraniach Recording-A/B/C (sekcja 6). Należy zauważyć, że analizowane pliki audio miały długość 2-3s a wysyłane przez AudioVerse ramki w trybie live mają 200ms, więc latencja jest również odpowiednio zawyżona na powyższej tabeli. Źródło: własne, zrzut ekranu raportu PDF generowanego przez* `POST /api/karaoke/lab/report/pdf`.[57]

Interpretacja: Na syntetycznym sygnale CREPE osiągnął RMSE 619,3 c wobec 629,3 c pYIN — przewaga jest niewielka (Δ = 10,0 c), co wskazuje na zbliżoną skuteczność obu algorytmów przy dostępie do wartości referencyjnej. W przypadku rzeczywistych nagrań, gdzie brak jest „wzorcowego" F0, metryki opisują spójność wzajemną: najwyższa zgodność wystąpiła dla Recording-A (RMSE 22,5 c, Acc@50c 95,6%), najniższa dla Recording-C (RMSE 49,8 c, Acc@50c 88,5%), co koreluje z przypuszczalnie większą złożonością akustyczną nagrania C. Latencja pYIN (362–396 ms) jest konsekwentnie 2,5–3,1× niższa niż CREPE (930–1207 ms), co przekłada się bezpośrednio na responsywność systemu w trakcie rozgrywki.

[57] Źródło ilustracji: Źródło własne — wykres z raportu PDF (POST /api/karaoke/lab/report/pdf), audioverse-react/src/pages/admin/lab/LabPage.tsx, dostęp: 2026-02-20.



Rysunek 5.3 — *Zbliżenie na wykres trajektorii F0 (2–3 s): CREPE (czerwona, gładka), pYIN (niebieska, z saltacją oktawową) i referencja (szara przerywana). Zaznaczony moment błędu oktawowego pYIN i poprawne wygładzenie CREPE dekoderem Viterbi. Źródło: własne, wykres wygenerowany z endpointa* `POST /api/karaoke/lab/pitch/compare`*, eksport z raportu PDF.*[58]

5.4. Ocena jakości systemu oceniania

Subiektywna spójność systemu oceniania badana była przez pięciu użytkowników testowych. Każdy wykonał tę samą piosenkę trzykrotnie z rosnącą starannością (celowo fałszując, śpiewając swobodnie, śpiewając możliwie poprawnie). Wyniki:

- Celowe fałszowanie: Score (μ = 1 234 pkt), Hits (μ = 12%), Perfect (μ = 3%).
- Swobodny śpiew: Score (μ = 4 871 pkt), Hits (μ = 54%), Perfect (μ = 22%).
- Staranny śpiew: Score (μ = 7 490 pkt), Hits (μ = 78%), Perfect (μ = 41%).

Wyniki są monotoniczne i statystycznie istotnie różne (test Kruskal-Wallis, $p < 0{,}01$), co potwierdza, że system różnicuje jakość wykonania zgodnie z subiektywną oceną śpiewu.

[58] Źródło ilustracji: Źródło własne — wykres trajektorii F0 z endpointa POST /api/karaoke/lab/pitch/compare, audioverse-react/src/pages/admin/lab/LabPage.tsx, dostęp: 2026-02-20.

5.5. Ograniczenia i kierunki dalszego rozwoju

Zidentyfikowane ograniczenia:

- Algorytm CREPE wymaga GPU dla akceptowalnej latencji w trybie live. Wdrożenie na serwerze bez karty graficznej NVIDIA powoduje ~8× wzrost czasu do ~2,5 s, co całkowicie wyklucza zastosowanie real-time.
- Konfiguracja środowiska GPU jest nietrywialnie złożona: dla różnych kart graficznych wymagane są różne wersje sterowników NVIDIA, CUDA i biblioteki torchcrepe. Obecna konfiguracja projektu przetestowana jest na karcie NVIDIA RTX 5060 Ti 16 GB; nie ma gwarancji, że system bez modyfikacji uruchomi się poprawnie na starszym sprzęcie.
- Nawet z GPU latencja CREPE jest subiektywnie zauważalna przez gracza, choć mieści się w granicy akceptowalności dla rozgrywki.
- Brak automatycznej kalibracji opóźnienia mikrofonu: przy opóźnieniu
- Biblioteka UltraStar TXT obsługuje standardowy wariant formatu; pliki sprzed 2007 roku z przestarzałymi nagłówkami mogą wymagać korekty nagłówka `#BPM:`.

Kierunki dalszego rozwoju:

- Fine-tuning CREPE na dedykowanym zestawie nagrań głosów amatorskich w celu poprawy RPA dla śpiewaków niewytrenowanych.
- Implementacja port WASM (`crepe.wasm`) do detekcji pitch po stronie przeglądarki — eliminacja latencji sieciowej.
- Rozbudowa silnika oceny o analizę dykcji (ASR — Automatic Speech Recognition) w celu oceny zgodności wymawianego tekstu.
- Automatyczna augmentacja istniejących plików UltraStar o nuty dostosowane do różnych poziomów trudności metodą quantization.
- Równoległy wariant aplikacji desktopowy lub mobilny (np. Flutter), który utylizowałby zasoby sprzętowe użytkownika — GPU karty konsumenckiej lub NPU — do uruchomienia CREPE lokalnie. Takie podejście pozwoliłoby zachować pYIN jako backend rozgrywki real-time w wersji webowej, jednocześnie oferując pełną analizę CREPE + DTW jako tryb pogłębionej analizy post-hoc bez kosztów infrastruktury po stronie serwera.

5.6. Podsumowanie i wnioski

System AudioVerse zrealizował zakładane cele pracy inżynierskiej. Przeprowadzona analiza literatury (rozdział 2) obejmuje przegląd ewolucji metod detekcji F0, charakterystykę algorytmów pYIN i CREPE oraz porównanie istniejących aplikacji karaoke — żadna z nich nie łączy automatycznej oceny wokalu opartej na deep learningu z webowym interfejsem działającym bez instalacji. Projekt i implementacja (rozdziały 3–4) obejmują pełnostosową architekturę mikroserwisową z dedykowanym modułem laboratoryjnym (`LaboratoryController`, sekcja 4.1.4), który umożliwił mierzalne testy porównawcze opisane w sekcji 5.3.

Przy wymaganiach systemu karaoke działającego w czasie rzeczywistym w przeglądarce optymalnym wyborem do detekcji pitch jest algorytm **pYIN** (latencja 78 ms < 150 ms na CPU, spełniony wymóg WNF01 w testowanym środowisku CPU). Wybór ten jest spójny z filozofią aplikacji karaoke jako gry couch — narzędzia dobrej zabawy towarzyskiej z elementem współzawodnictwa, nie studia nagrań wymagającego studyjnej perfekcji. CREPE stanowi uzupełnienie o wyraźnie wyższej dokładności (RMSE niższy o ~38% dla czystego wokalu, ~48% dla miksu) i jest zalecany jako tryb post-hoc oraz jako opcjonalny tryb live dla instalacji z GPU. Warunkiem jest odpowiedni sprzęt — konfiguracja zależności GPU wymaga dopasowania do konkretnej karty graficznej, co utrudnia jednak opracowanie uniwersalnej metody wdrażania.

Zbudowany system AudioVerse spełnia zarówno funkcję rozrywkową (gra karaoke z oceną), jak i pomiarową (platforma testowa umożliwiająca mierzalne porównanie algorytmów AI na plikach audio). Stanowi dowód, że wysoka jakość automatycznej analizy śpiewu nie wymaga dziś ani specjalistycznego sprzętu, ani zamkniętej platformy: wystarczy przeglądarka i mikrofon.

Bibliografia

Książki i monografie

[1] M. Müller, Fundamentals of Music Processing: Using Python and Jupyter Notebooks, wyd. 2, Springer, 2021.

[2] A. Lerch, An Introduction to Audio Content Analysis: Applications in Signal Processing and Music Informatics, Wiley, 2012.

[3] T. Zieliński, Cyfrowe przetwarzanie sygnałów. Od teorii do zastosowań, Wydawnictwo Komunikacji i Łączności, 2009.

[4] T. Mitsui, S. Hosokawa (red.), Karaoke Around the World: Global Technology, Local Singing, Routledge, 1998.

[5] R. Drew, Karaoke Nights: An Ethnographic Rhapsody, AltaMira Press, 2001.

Artykuły naukowe i konferencyjne

[6] M. Mauch, S. Dixon, "pYIN: A Fundamental Frequency Estimator Using Probabilistic Threshold Distributions", Proc. ICASSP, s. 659–663, 2014.

[7] J. W. Kim, J. Salamon, P. Li, J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation", Proc. ICASSP, 2018.

[8] A. Défossez, "Hybrid Spectrogram and Waveform Source Separation", Proc. ISMIR 2021, arXiv:2111.03600, 2021.

[9] J. Salamon, E. Gómez, "Melody Extraction from Polyphonic Music Signals using Pitch Contour Characteristics", IEEE Trans. ASLP, vol. 20, nr 6, s. 1759–1770, 2012.

[10] A. de Cheveigné, H. Kawahara, "YIN, a fundamental frequency estimator for speech and music", J. Acoust. Soc. Am., vol. 111, nr 4, s. 1917–1930, 2002.

[11] E. J. Humphrey, J. P. Bello, Y. LeCun, "Moving Beyond Feature Design: Deep Architectures and Automatic Feature Learning in Music Informatics", Proc. ISMIR, 2012.

[12] F.-R. Stöter, S. Uhlich, A. Liutkus, Y. Mitsufuji, "Open-Unmix — A Reference Implementation for Music Source Separation", JOSS, vol. 4, nr 41, 2019.

[13] G. F. Welch i in., "The Impact of Singing on Brain and Behaviour", Progress in Brain Research, vol. 237, 2018.

[14] H. Sakoe, S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition", IEEE Trans. ASSP, vol. 26, nr 1, s. 43–49, 1978.

Spis ilustracji

Rysunek 2.1. Interfejs gry SingStar (PlayStation 2): pasek nut z kursorem śpiewaka i bieżącą punktacją. Stanowi wzorzec interfejsu karaoke z oceną pitch. Źródło: materiały prasowe Sony Computer Entertainment Europe (screeny dostępne w serwisach gamepress.gg i gaming.wikia.com).

Rysunek 2.2. Ekran rozgrywki UltraStar Deluxe: timeline nut z sylabami, kursor pitch gracza i bieżąca punktacja. Interfejs wzorowany na SingStar, open-source z pełną obsługą formatu UltraStar TXT. Źródło: materiały z oficjalnej strony projektu UltraStar Deluxe (usdx.eu).

Rysunek 2.3. Interfejs Smule w trybie duetowym (Join): podświetlony tekst piosenki, warstwa wideo z kamery, brak wskaźnika pitch. Ocena opiera się wyłącznie na metrykach społecznościowych (lajki, komentarze). Źródło: materiały prasowe Smule (smule.com).

Rysunek 2.4. Interfejs KaraFun Desktop: licencjonowana baza piosenek, tryb DJ z kolejkowaniem przez publiczność i synchronizowany tekst. Brak jakiejkolwiek analizy wokalu lub oceny intonacji. Źródło: materiały z oficjalnej strony KaraFun (karafun.com).

Rysunek 2.5. Przeglądarka piosenek AudioVerse z filtrowaniem po tytule, artyście, języku i gatunku. Widoczna lista kafelków piosenek z metadanymi, paginacja i skeleton loadery podczas ładowania. Źródło: własne, zrzut ekranu aplikacji AudioVerse, widok KaraokeSongBrowserPage (`audioverse-react/src/pages/party/KaraokeSongBrowserPage.tsx`).

Rysunek 2.6. Spektrogram STFT (lewy) i spektrogram Mel (prawy) 3-sekundowego fragmentu wokalnego. Oś Y lewego: częstotliwość [Hz]; oś Y prawego: skala melowa (128 filtrów). Widoczne harmoniczne głosu jako jaśniejsze poziome linie. Źródło: własne, wygenerowany skryptem Python (librosa + matplotlib) na przykładowym nagraniu wokalnym (mono, 16 kHz).

Rysunek 2.7. Porównanie trajektorii F0 wykrytych przez pYIN i CREPE dla tego samego 5-sekundowego pliku wokalnego. Oś Y: częstotliwość [Hz], oś X: czas [s]. Linia niebieska — pYIN, czerwona — CREPE, szara — referencja. Źródło: własne, wygenerowany przez endpoint `POST /api/karaoke/lab/pitch/compare`, wizualizacja z raportu PDF zwracanego przez `POST /api/karaoke/lab/report/pdf`.

Rysunek 3.1. Schemat komunikacji między warstwami systemu AudioVerse: przeglądarka → ASP.NET API → mikroserwisy AI (FastAPI) → PostgreSQL / MinIO. Dla każdego mikroserwisu podano protokół (HTTP/WebSocket) i średnią latencję. Źródło: własne, diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/01-system-architecture.drawio`.

Rysunek 3.2. Diagram encji modelu danych modułu karaoke (ER). Widoczne relacje między KaraokeSession, KaraokeSessionRound, KaraokeSessionRoundPlayer i KaraokeSinging. Źródło: własne, wygenerowany diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/auto-data-model.drawio`, wygenerowany narzędziem `AudioVerse.DiagramGenerator`.

Rysunek 4.1. Diagram architektury CQRS w AudioVerse: Controller → MediatR → Handler → Repository → DbContext → PostgreSQL. Widoczne przykłady: SaveKaraokeSingingCommand → SaveKaraokeSingingHandler (zapis wyników) i GetRankingsQuery → GetRankingsHandler (odczyt rankingów). Źródło: własne, diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/08-cqrs-architecture.drawio`.

Rysunek 4.2. Diagram przepływu sesji karaoke: CreateSession → kod dołączenia → JoinByCode → CreateRound → AssignPlayers → strumieniowanie PCM → detekcja pitch → zapis wyników → RoundFinished → ekran rankingu. Źródło: własne, diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/09-karaoke-session-flow.drawio`.

Rysunek 4.3. Diagram przepływu autoryzacji JWT i OAuth 2.0 w AudioVerse: Login → Identity Server → JWT Bearer → SignalR; przepływ OAuth (Google / Spotify / Twitch / Discord) → callback → token refresh. Role: User i Admin z oddzielnymi uprawnieniami do endpointów. Źródło: własne, diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/06-auth-jwt-flow.drawio`.

Rysunek 4.4. Swagger UI endpointu modułu laboratoryjnego (`/api/karaoke/lab`). Widoczne wszystkie 9 endpointów z opisami, formatów wejścia/wyjścia i możliwością testowania bezpośrednio z przeglądarki. Źródło: własne, zrzut ekranu Swagger UI dostępnego pod adresem `http://localhost:5000/swagger` po uruchomieniu `docker compose up -d`.

Rysunek 4.5. Diagram pipeline modułu laboratoryjnego: Upload WAV → LaboratoryController → audio_separate (Demucs) → audio_pitch (pYIN/CREPE) →

ComputeMetrics (RMSE, Acc@50c, Pearson r) → sing_score (DTW) → LaboratoryReportPdfService (QuestPDF) → odpowiedź PDF. Źródło: własne, diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/13-laboratory-pipeline-flow.drawio`.

Rysunek 4.6. Struktura routingu aplikacji React/TypeScript: RootLayout z trasami publicznymi (/songs, /ranking, /play, /wiki), AuthLayout (/login, /register) i AdminLayout (/admin/dashboard, /admin/lab). Widoczna hierarchia layoutów z lazy-loadingiem. Źródło: własne, diagram oparty na konfiguracji z `audioverse-react/src/App.tsx`.

Rysunek 4.7. Przeglądarka piosenek (KaraokeSongBrowserPage): panel filtrowania po tytule, artyście, języku i gatunku; siatka kafelków z okładką, tytułem i artystą; paginacja wyników. Źródło: własne, zrzut ekranu aplikacji AudioVerse, widok KaraokeSongBrowserPage (`audioverse-react/src/pages/party/KaraokeSongBrowserPage.tsx`).

Rysunek 4.8. Ekran rozgrywki karaoke (KaraokeRoundPage): pełnoekranowy Canvas 1920×1080 px z timeline nut, kursorem pitch (linia pionowa), aktywną sylabą, wskaźnik score i combo w prawym górnym rogu. Źródło: własne, zrzut ekranu aplikacji AudioVerse, widok KaraokeRoundPage (`audioverse-react/src/pages/party/KaraokeRoundPage.tsx`).

Rysunek 4.9. Ekran rankingu po zakończeniu piosenki (KaraokeRankingPage): tabela graczy posortowana malejąco po Score, kolumny Hits / Misses / Good / Perfect / Combo, animacja wejścia rekordów i przycisk przejścia do następnej rundy. Źródło: własne, zrzut ekranu aplikacji AudioVerse, widok KaraokeRankingPage (`audioverse-react/src/pages/party/KaraokeRankingPage.tsx`).

Rysunek 4.10. Widok tworzenia sesji karaoke (formularz z kodem dołączenia, lista piosenek, przycisk „Utwórz sesję") i ekran dołączania do sesji (pole na kod, przycisk „Dołącz"). Źródło: własne, zrzuty ekranu widoków PartiesPage i JoinPartyPage (`audioverse-react/src/pages/party/`).

Rysunek 4.11. Strona ustawień wejścia audio: lista mikrofonów, wizualizacja poziomu sygnału (VU meter), wybór algorytmu detekcji pitch (pYIN / CREPE) i przycisk testu mikrofonu. Źródło: własne, zrzut ekranu widoku AudioInputSettingsPage (`audioverse-react/src/pages/settings/AudioInputSettingsPage.tsx`).

Rysunek 4.12. Strona modułu laboratoryjnego w panelu admina (/admin/lab): zakładki Pitch, Separation, Score, Benchmark, Report; formularz uploadu pliku WAV; wyniki analizy z metrykami, trajektoriami F0 i wskaźnikami statusu mikroserwisów AI (zielone/czerwone). Źródło: własne, zrzut ekranu widoku LabPage (`audioverse-react/src/pages/admin/lab/LabPage.tsx`).

Rysunek 4.13. Diagram kontenerów Docker Compose systemu AudioVerse: 10 serwisów (API, Identity, 5 × AI FastAPI, PostgreSQL, MinIO, Redis) z oznaczonymi portami i zależnościami sieciowymi. Źródło: własne, diagram dostępny w pliku `AudioVerse.API/Docs/diagrams/11-docker-containers.drawio` (otworzyć w draw.io lub VS Code z rozszerzeniem Draw.io Integration).

Rysunek 5.1. Wyniki testów jednostkowych i integracyjnych systemu AudioVerse: konsola Vitest (frontend React/TypeScript) z wszystkimi testami zaliczonymi oraz xUnit (backend C#) ze statystyką wykonania. Źródło: własne, zrzut ekranu konsoli po `npx vitest run` i `dotnet test`.

Rysunek 5.2. Zbiorczy widok raportu modułu laboratoryjnego: wykres słupkowy średniej latencji (sekcja 4), tabela kalibracji na syntetycznym sygnale referencyjnym (sekcja 5) oraz tabela porównania CREPE i pYIN na rzeczywistych nagraniach Recording-A/B/C (sekcja 6). Należy zauważyć, że analizowane pliki audio miały długość 2-3s a wysyłane przez AudioVerse ramki w trybie live mają 200ms, więc latencja jest również odpowiednio zawyżona na powyższej tabeli. Źródło: własne, zrzut ekranu raportu PDF generowanego przez `POST /api/karaoke/lab/report/pdf`.

Rysunek 5.3. Zbliżenie na wykres trajektorii F0 (2–3 s): CREPE (czerwona, gładka), pYIN (niebieska, z saltacją oktawową) i referencja (szara przerywana). Zaznaczony moment błędu oktawowego pYIN i poprawne wygładzenie CREPE dekoderem Viterbi. Źródło: własne, wykres wygenerowany z endpointa `POST /api/karaoke/lab/pitch/compare`, eksport z raportu PDF.

Spis tabel

Tabela 2.1. Porównanie aplikacji karaoke

Tabela 2.2. Porównanie algorytmów detekcji F0

Tabela 3.1. Wymagania funkcjonalne (WF)

Tabela 3.2. Encje domeny karaoke

Tabela 3.3. Stos technologiczny AudioVerse

Tabela 4.1. Endpointy modułu laboratoryjnego (/api/karaoke/lab)

Tabela 5.1. Wyniki porównania pYIN vs CREPE